CAPSTONE PROJECT REPORT

(Project Term January–May 2026)

PROXYMKT: A CLOUD-NATIVE HYBRID ATTENDANCE SYSTEM

Submitted by

Sameer Mahato Registration Number: 12202633

Anjeet Patel Registration Number: 12210752

Sumant Yadav Registration Number: 12201495

Janhavi Tak Registration Number: 12211379

Hritu Raj Registration Number: 12200402

Project Group Number: 2RGC0375

Course Code: INT439

Under the Guidance of

Neha Sandotra, Assistant Professor

School of Computer Science and Engineering

PAC FORM

TOPIC APPROVAL PERFORMA

School of Computer Science and Engineering

Program : P132::B.Tech. (Computer Science and Engineering)

COURSE CODE : CSE439 **REGULAR/BACKLOG :** Regular **GROUP NUMBER :** 2RGC0375

Supervisor Name : Neha **UID :** 33904 **Designation :** Assistant Professor

Qualification : _____ **Research Experience :** _____

SR.NO.	NAME OF STUDENT	Prov. Regd. No.	BATCH	SECTION
1	Janhavi Tak	12211379	2022	422GN
2	Anjeet Patel	12210752	2022	422UN
3	Sameer Mahato	12202633	2022	422XG
4	Sumant Yadav	12201495	2022	222SF
5	Hritu Raj	12200402	2022	322AK
6	Hritu Raj	12200402	2022	322AK

SPECIALIZATION AREA : Cloud Computing **Supervisor Signature:** _____

PROPOSED TOPIC : A Smart Hybrid Attendance System

Qualitative Assessment of Proposed Topic by PAC		
Sr.No.	**Parameter**	**Rating (out of 10)**
1	Project Novelty: Potential of the project to create new knowledge	6.50
2	Project Feasibility: Project can be timely carried out in-house with low-cost and available resources in the University by the students.	7.25
3	Project Academic Inputs: Project topic is relevant and makes extensive use of academic inputs in UG program and serves as a culminating effort for core study area of the degree program.	6.75
4	Project Supervision: Project supervisor's is technically competent to guide students, resolve any issues, and impart necessary skills.	7.25
5	Social Applicability: Project work intends to solve a practical problem.	7.25
6	Future Scope: Project has potential to become basis of future research work, publication or patent.	7.25

PAC Committee Members		
PAC Member (HOD/Chairperson) Name: Dr. Manjit Kaur	UID: 12438	Recommended (Y/N): Yes
PAC Member (Allied) Name: Aarti Shar	UID: 29559	Recommended (Y/N): Yes
PAC Member 3 Name: Priyanka Gotter	UID: 29530	Recommended (Y/N): Yes

Final Topic Approved by PAC: **A Smart Hybrid Attendance System**

Overall Remarks: Approved

PAC CHAIRPERSON Name: 16479::Dr. Parminder Singh **Approval Date:** 27 Apr 2026

4/27/2026 11:35:54 AM

DECLARATION

We hereby declare that the project work entitled ("ProxyMukt: Intelligent Multi-layer Attendance System to Prevent Proxy") is an authentic record of our own work carried out as requirements of Capstone Project for the award of B.Tech degree in Computer Science and Engineering from Lovely Professional University, Phagwara, under the guidance of Neha Sandotra, during January to May 2026. All the information furnished in this capstone project report is based on our own intensive work and is genuine.

Project Group Number: 2RGC0375

Sameer Mahato	Sameer Mahato
12202633	Date: 29/04/26
Anjeet Patel	Anjeet Patel
12210752	Date: 29/04/26
Sumant Yadav	Sumant Yadav
12201495	Date: 29/04/26
Janhavi Tak	Janhavi Tak
12211379	Date: 29/04/26
Hritu Raj	Hritu Raj
12200402	Date: 29/04/26

CERTIFICATE

This is to certify that the declaration statement made by this group of students is correct to the best of my knowledge and belief. . The Capstone Project has been done by them under my supervision. The work is their own original work, effort and study. No part of the work has been submitted for any other degree at any University. The Capstone Project is suitable for submission for partial fulfilment of the requirements for the degree of B.Tech in Computer Science & Engineering at Lovely Professional University, Phagwara.

Neha Sandotra

Signature and Name of the Mentor: Neha Sandotra
Designation: Assistant Professor

School of Computer Science and Engineering,
Lovely Professional University,
Phagwara, Punjab.

Date: 29/04/26

ACKNOWLEDGEMENT

We would like to express our gratitude to our faculty mentor at the school of Computer Science and Engineering Lovely Professional University who has been there all along, provided timely feedback and peer mentoring in the development of ProxyMukt. The technical intricacies that were achieved in this project in the design and construction of cryptographic tokens, real time communication and the multiple layer fraud detection was possible because of the mentoring and guidance provided to us at every step. We also would like to thank the Department of Computer Science and Engineering for providing us with the facilities in terms of development infrastructure, cloud platform development tools and the ability to work in an academic environment to complete a 18 week development cycle. The emphasis on documentation and security conscious design that was the result of design choices in the project. We would also like to give a big shout-out to the open-source community whose frameworks and tools are the foundation for ProxyMukt. This also extends to the developers of MongoDB, express.js, React.js, Node.js, Socket.Io, bcrypt and Helmet.js. We also recognize IETF working groups whose RFC documents (especially RFC 2104 (HMAC)) and RFC 7519 (JWT)) supplied the cryptographic standards our verification pipeline is based upon.

The student and faculty volunteers that took part in the adversarial pilot testing sessions are also appreciated. Their eagerness to model attack scenarios like GPS spoofing, VPN masking, and QR screenshot replay, allowed us to tune the risk thresholds of the pipeline to real-world institutional constraints and contributed greatly to the final system reliability.

Lastly, we would like to thank our families and friends who were patient, encouraging and supported us during the long period of development and documentation of this project. They inspired us to continue in tough times of the project.

TABLE OF CONTENTS

Inner first page…….(i)
PAC form……(ii)
Declaration…...……..(iii)
Certificate……..……..……(iv)
Acknowledgement……...…….....(v)
Table of Contents……...……(vi)

1. INTRODUCION 1
1.1 Background and Motivation 2
1.2 System Overview and Stakeholder Roles 3
1.3 Technology Stack Summary 4
1.4 Learning Outcomes Alignment 5
1.5 Objectives of the Project 5
1.6 Scope of the Project 6
1.7 Organization of the Report 6
2. PROFILE OF THE PROBLEM, RATIONALE, AND SCOPE 7
2.1 PROBLEM PROFILE 7
2.2 RATIONALE 8
2.3 Current Challenges in Attendance Systems 8
3. EXISTING SYSTEM 11
3.1 INTRODUCTION TO EXISTING SYSTEMS 11
3.2 EXISTING SOFTWARE AND APPROACHES 11
3.2.1 Manual Roll-Call Systems 11
3.2.2 RFID and NFC Card Systems 12
3.2.3 Static QR Code Systems 13
3.2.4 Location-Based Geofencing Systems 12
3.2.5 Facial Recognition Systems 13
3.2.6 Online and Hybrid Attendance Models 13
3.3 DATA FLOW DIAGRAM — EXISTING SYSTEM 14
3.4 Limitations of Existing Systems 14
3.5 What is New in the Proposed System 15

4. PROBLEM ANALYSIS 17
4.1 PRODUCT DEFINITION 17
4.2 FEASIBILITY ANALYSIS 18
4.2.1 Technical Feasibility 19
4.2.2 Economic Feasibility 19
4.2.3 Operational Feasibility 20
4.3 PROJECT PLAN 20
5.SOFTWARE REQUIREMENT ANALYSIS 23
5.1 INTRODUCTION 23
5.2 GENERAL DESCRIPTION 23
5.2.1 Operating Environment 23
5.2.2 User Classes and Characteristics 23
5.2.3 Constraints 24
5.3 SPECIFIC REQUIREMENTS 24
5.4 TECHNOLOGY STACK SPECIFICATION 26
5.5 Hardware and Software Requirements 27
5.5.1 Server Requirements 27
5.5.2 Client Requirements 27
5.5.3 Development Tools 27
6. SYSTEM DESIGN 29
6.1 SYSTEM ARCHITECTURE 29
6.2 ARCHITECTURE DIAGRAM 30
6.3 SIX-LAYER VERIFICATION PIPELINE DESIGN 31
6.4 DATABASE DESIGN 33
6.5 API ENDPOINT DESIGN 35
6.6 FLOWCHARTS 36
6.7 HYBRID ATTENDANCE MODEL DESIGN 37
7. TESTING 38
7.1 TESTING STRATEGY 38
7.2 FUNCTIONAL TESTING 38
7.3 STRUCTURAL TESTING 42
7.4 LEVELS OF TESTING 43

7.4.1 Unit Testing 43
7.4.2 Integration Testing 43
7.4.3 System Testing 43
7.4.4 Regression Testing 44
7.5 Adversarial Test Cases and Results 44
8. IMPLEMENTATION 47
8.1 IMPLEMENTATION OF THE PROJECT 47
8.2 SYSTEM IMPLEMENTATION 47
8.3 Frontend Implementation 47
8.4 Real-Time Integration (Socket.IO) 48
8.5 Conversion Plan 48
8.6 Post-Implementation and Software Maintenance 49
9. PROJECT LEGACY 51
9.1 Current Status of the Project 51
9.2 Remaining Areas of Concern 53
9.3 Technical and Managerial Lessons Learnt 54
9.3.1 Technical Lessons 54
9.3.2 Managerial Lessons 55
10: USER MANUAL 57
10.1 System Access and Registration 57
10.2 Student User Guide 57
10.2.1 Accessing the Student Dashboard 57
10.2.2 Marking Attendance — Offline Session 58
10.2.3 Online Session Attendance 58
10.2.4 Viewing Attendance History and Analytics 59
10.3 Faculty User Guide 59
10.3.1 Creating and Managing Classes 59
10.3.2 Starting and Managing Live Sessions 59
10.3.3 Viewing Analytics and Exporting Reports 60
10.3.4 Verification Layer Configuration 60
10.4 Administrator User Guide 61
10.5 Troubleshooting Guide 61

11. SOURCE CODE REFERENCE AND SYSTEM SNAPSHOTS 63

11.1 Repository Structure 63

11.2 Critical Code Excerpts 64

11.3 System Snapshots Description 64

12. BIBLIOGRAPHY 69

1. INTRODUCION

The attendance monitoring is one of the fundamental functioning requirements of education facilities worldwide. Its implications go way beyond record-keeping: the adherence to policies, the entitlement of a student to take tests and to be given financial aid, the institutional funding requirements, and the prompt identification of students who are academically at risk is directly reliant upon the quality and integrity of attendance data. Even though this is the core of this role, in practice, most institutional attendance systems are still paper-based or of primitive digital implementation which offer only a minor advantage in security or efficiency.

Even though paper-based roll-call registers are universal to understand, need some instructional time to operate - even conservatively 810 minutes/class period - and create transcription errors, they are virtually silent without a centralised data layer. More to the point, they are quite likely to commit a proxy attendance, which is one of the most frequently reported types of academic dishonesty, as a student fakes an attendance on behalf of one of his or her peers. The widespread availability of digital attendance solutions (including the most common static QR code) and RFID cards or simply location-based check-ins have not helped significantly to mitigate this vulnerability. Mobile applications that are free to download and use can easily overcome screenshot sharing, credential delegation, and card sharing and location spoofing techniques.

The verification landscape has also been augmented by the introduction of hybrid and online learning formats that have since 2020 been accelerated by world events. The geolocation-based attendance is based on physical distance cues that cannot be applied to virtual classes. Passivity online - a student whose browser window is open or has access link open, but is not actively involved in any system currently in use, is in fact the same as active participation. The resultant effect is a methodical loss of integrity of attendance information within the very learning environments where student engagement monitoring is the most profound.

These conflicting forces in architecture are said to have led to proxymukt. It is a real-time Socket, hardware-agnostic, cloud-native attendance engine which was created on the top of the MERN stack (MongoDB, Express.js, React.js, Node.js).IO communication. It proposes a 6-layer sequential verification pipeline, cryptographic

HMAC-SHA256 rotating and geofencing QR with the Haversine formula, anti-spoofing of devices by the GPS, proxy/VPN risk analysis on the network level, a database-based device fingerprinting and duplicate prevention. It is a multi-layered design which is clearly designed to introduce cost and complexity to the informal proxy fraud and can be implemented on student hardware commoditized and can be scaled to institutional scales, without necessarily needing to deploy a biometric infrastructure.

1.1 Background and Motivation

One of the most basic administrative functions in any learning institution is the attendance tracking. It plays a vital role in decision making on the eligibility of a student to sit for exam in a given case, payment of his/her scholarship, evaluation of his/her academic performance and the compliance of the institution to regulatory authorities. It is an important information yet the attendance data of majority of institutions are compromised by a problem known as proxy attendance, where students will be recording attendance for others who are absent.

This problem has existed for some time, but has been increasing as the use of online attendance systems becomes more common. In particular, the QR code-based attendance systems that have become a popular choice owing to their ease of use and affordability of implementation have a major weakness; the QR code is just an image that can be scanned and shared via a texting software. As a student scans the QR code, he can send a picture of the QR code to another student that is not at the class and he can then mark the attendance from any location in the world. Similarly, spoofing apps that are free for Android devices can be used to spoof location-based attendance systems and physical cards can be used to spoof RFID-based attendance systems.

The other problem is that most of the existing systems are based on a single verification process. If this single path is compromised, then this will affect the attendance record. The faculty is given attendance sheets that seem valid, but contain invalid data which eventually gets transferred to academic reporting, scholarship, and course credit calculations based on attendance.

This was the motivation for ProxyMukt. In the pilot study at Lovely Professional University, it was realized that one motivated student was able to check the attendance of three other non-present students in a normal one-hour class session

with just a screenshot of the QR code and a messaging application - it is not necessary to have any technical expertise at all. This indicated a serious lapse between what the current attendance systems will purport to check and what they will check.

The ProxyMukt was created directly in reaction to these compounding vulnerabilities. The system builds a sequential six-layer verification pipeline, instead of a one-check verification signal, with every layer processing a particular and different fraud vector. The idea is to increase the combined cost and difficulty to commit fraud that most students would not be willing to risk their attempts to do so to get an attendance record.

1.2 System Overview and Stakeholder Roles

ProxyMukt is a complete web application, which is developed based on MERN stack MongoDB, Express.js, React.js and Node.js with Socket.IO that offers real-time two-way communication between the server and clients that are connected to it. It is a cloud-native system, i.e. is able to run on fully managed cloud infrastructure without any on-prem hardware. The system can be accessed via any modern web browser and does not require any installation of client-side software and is hence very easy to adopt. The system has three major roles, which have their own dashboards and roles defined:

Student Role: Students are the major users of the system as they are the ones who mark their attendance during the classes. They can access their personal dashboard to scan the QR code provided by the faculty member, get verified and view their attendance history, percentage, and other analytics. They also receive push notifications in real-time if they are not attending according to the institute's rules.

Faculty Role: Faculty can create classes and sessions from their dashboard. They can create classes and add students, establish and manage geofence radii of classes, start and end live classes, and real-time class attendance monitoring as students check-in. Faculty dashboard also contains class-wise analytics like identification of vulnerable students in a class, and the distribution of risk within the class.

Administrator Role: Administrators are able to access institution-level governance tools. It has a cross-class attendance heatmap, full access to audit logs (filterable by student, faculty, session, or event type), user account management, geofence calibration tools on all registered teaching spaces, and a system health monitoring dashboard.

1.3 Technology Stack Summary

The technology decisions related to ProxyMukt were based on three priorities, namely: security, performance and deployment ease. The technologies selected are all open-source, are well-documented and have been tested in production environments.

Table 1: Technology Stack Summary

Layer	Technology	Justification
Frontend	React 18.x + Vite	Component-based SPA with hook-driven WebSocket state management
Styling	Tailwind CSS + Material UI	Responsive design with accessible, pre-built component library
Backend	Node.js 20 LTS + Express.js	Non-blocking I/O suited for high-concurrency session management
Real-Time	Socket.IO 4.x	Bidirectional WebSocket communication with sub-second QR broadcast
Authentication	JSON Web Token (Json web token 9.x)	Short-lived access tokens with refresh token rotation
Cryptography	Node.js Crypto (HMAC-SHA256)	FIPS 198-1 compliant, no external dependency required
Password Hashing	bcrypt 5.x	Adaptive cost-factor hashing with a minimum of 10 salt rounds
Database	MongoDB Atlas 7.x	Flexible schema with compound index support for duplicate prevention
Security Middleware	Helmet + Express Rate Limit	HTTP security headers and denial-of-service mitigation

1.4 Learning Outcomes Alignment

This capstone project is consistent with the programme-level learning outcomes (PLOs) of the Lovely Professional University Tech Computer Science and Engineering

programme.

PLO-1 (Engineering Knowledge) is illustrated by the use of cryptographic algorithms like HMAC-SHA256, geospatial algorithms like Haversine formula, as well as the principles of distributed systems architecture. The systematic identification of six different attendance fraud vectors and their formalization into a layered threat model attests to PLO-2 (Problem Analysis). The implementation of the six-layer verification pipeline and the entire MERN application realizes PLO-3 (Design and Development of Solutions). PLO-4 (Investigation): is evidenced by adversarial pilot testing and performance evaluation in a structured form. PLO-5 (Modern Tool Usage) is the entire technology stack used in an actual production environment. PLO-6 (Engineer and Society) is covered by the analysis of privacy impact, data minimization choice, and consideration of fairness in the creation of automated monitoring mechanisms of students.

1.5 Objectives of the Project

The main goals of the ProxyMukt system are the following: To remove proxy attendance by deploying a multi-layer verification pipeline that can handle multiple fraud vectors at the same time. To give faculty real-time access to attendance during live sessions, allowing them to be aware of suspicious activity as soon as possible. To decrease the administrative workload on faculty through automatizing attendance monitoring, report building, and alerts. To deploy industry-standard security features such as cryptographic token signing, JWT based authentication, bcrypt password hashing and audit logging. To facilitate offline (physical classroom) and online (hybrid) classes with the right verification logic in each modality. To create actionable analytics that can be used by faculty and administrators to determine which students are at risk and make decisions based on data about academic intervention. To protect student privacy by not retaining raw biometric data or facial images and by keeping the data collected to the minimum needed to verify. To develop a scaled, cloud-native architecture that could support a high number of parallel users and sessions without affecting the performance.

1.6 Scope of the Project

The scope of ProxyMukt covers the following areas:

In Scope: The system manages classroom and session attendance of both physical (offline) and online/hybrid learning approaches. It uses a six-layer sequential fraud detection pipeline that includes QR token freshness, physical proximity, GPS anti-spoofing, network-origin risk analysis, device fingerprinting and duplicate prevention. Student, Faculty and Administrator roles have role-specific dashboards and reporting. Operation of real time session such as sub-second QR token broadcast are enabled through Socket.IO. All the events of attendance and the pipeline decision points use tamper-evident audit logging. The system was tested adversarially against screenshot replay and GPS spoofing, VPN masking, credential sharing, and duplicate submission attack vectors.

Out of Scope: Its latest release lacks native iOS or Android mobile applications, persistent biometric or facial identity storage, indoor location-finding via Wi-Fi, Bluetooth, strong compatibility with existing Student Information Systems or Learning Management Systems, automation of disciplinary processes other than flagging and audit logging, and multi-institutional implementation other than the pilot institution.

1.7 Organization of the Report

This report is divided into twelve chapters. Chapter 2 contains the problem profile, justification and scope. Chapter 3 analyses the existing systems of attendance and the problems with them. Chapter 4 has the problem analysis, feasibility and project plan. Chapter 5 formalises the software requirements specification. The chapter 6 contains the complete system design including the system architecture, six-layer pipeline, database design, API contracts, flowcharts and pseudocode. In chapter 7, the test plan and results of an adversarial test are given. Chapter 8 will cover the implementation, source code and migration plan. Chapter 9 will discuss the project legacy, issues still to be resolved and lessons learnt. A copy of the user manual is provided in chapter 10. Chapter 11 discusses about the source code repository and interface snapshots of the system. The references are given in Chapter 12 in IEEE format.

2. PROFILE OF THE PROBLEM, RATIONALE, AND SCOPE

2.1 PROBLEM PROFILE

One of the most prevalent and least talked about cases of academic dishonesty in institutions of higher learning around the world is proxy attendance. The practice of recording attendance for another student creates a spurious record which seeps into institutional processes. The absentee student appears to be eligible to sit for exams, scholarships that require attendance as a criterion, and doesn't trigger any form of intervention measures that should be triggered by established systems based on the real absence data of the student. Research has consistently linked chronic absence to poor academic performance and high attrition and therefore the need for integrity in attendance is an institutional governance and a student welfare issue.

The proxy attendance structural enabler is the discrepancy between what is actually checked by the current systems and what they purport to check. A non-dynamic QR code system, e.g., ensures that a user is able to decode a QR token when it is scanned. It is not checking if the user is actually in the classroom or the scanning device is one the registered student has, or that the network point of the submission is physically in the classroom or the location of the same student is submitting a session of attendance. All these untested dimensions are attack surfaces that can be exploited with no technical skills.

To provide an example of how serious this gap can be, adversarial testing run during the ProxyMukt pilot at Lovely Professional University revealed that one student was capable of marking attendance on three other non-present students in the span of a regular classroom session with the help of a simple messaging program and a screenshot of the displayed QR code. The ability to perform and no technical skills, no special tools, and no access to any system other than a basic smartphone were needed. This is the reality on the ground that ProxyMukt will solve.

2.2 RATIONALE

Three overlapping remarks on the space of the design of a successful attendance fraud countermeasure that hint on the design of ProxyMukt.

First of all, it is possible to implement multi-signal checking with no special equipment. The newest generation of student smartphones already has all sensor and network features needed to implement multi-layer checking: it has a GPS receiver to get the location information, a browser API to enumerate the properties of the devices, a network interface the IP-address of which can be looked up in reputation databases. This model is also hardware neutral and eliminates the obstacle that is the capital expenditure that has limited the implementation of biometric or RFID-based technologies in resource-deprived institutions in the past.

Second, the layered design is a factor that makes single-factor systems inherently weak. Any one-factor system may be killed by the violation of one signal and this kills the chain of verification. ProxyMukt sequential pipeline is constructed in layers that removes this weakness by undertaking verification of all the layers to pass its own. The success of one of the layers does not abate the verification. The security of the entire pipeline is thus far greater than is the security provided by any of the components.

Third, cryptographic signature of QR tokens turns them into dynamic verification credentials and not an image only. ProxyMukt HMAC-SHA256-signs QR tokens based on the IETF RFC 2104, transforms the QR code into a token 20 seconds long that can be exchanged in the form of a picture. This change alone causes the screenshot replay attacks to be operationally infeasible in practically all real-world contexts.

2.3 Current Challenges in Attendance Systems

The attendance systems are currently in use in educational institutions and pose a vast array of challenges to the institutions. These challenges may be categorized as per the type of system.

The most common attendance system used in resource-poor settings are Paper-Based Systems. They do not require infrastructure, but they are very limited. Time spent on roll-call (on average) is eight to ten minutes per class per day. The paperwork

can be prone to errors in transcribing, being lost or damaged, and does not allow access to administrators in real-time. More importantly, a proxy manual system is merely a verbal recommendation by a current student there is no technological barrier to proxy marking.

The most common hardware-based electronic system and that is mostly used in well-resourced organisations is RFID and NFC Card Solutions. They are quick and efficient, but are single factor systems. It mainly confirms the presence of the card, not the student. Use of the proxy mailing concept is straight-forward and non-technical. In addition, it is expensive to deploy because of the cost of hardware to be installed per classroom.

Static QR Code Systems are cheap to set up and are widely adopted because they are easy to use. The primary weakness of the system is the tokens (QR) are non-reusable. Students can even take a snapshot of the exhibited QR code and send it to students absent via any chat software and allow them to mark their attendance from anywhere in the world. Lacking cryptographic freshness, temporal binding or location-based verification, these systems are not much more secure than attendance via word-of-mouth.

Geofencing Systems Location-Based Geofencing Systems add a location element to the attendance process. They will require students to provide the GPS location in a certain region at the time of roll call. While this will prevent any simple off-site cheating of the attendance, indoor GPS will be less accurate - as low as 10-30 metres for dense structures with limited sky-view - leading to the exclusion of some legitimate students who happen to be sitting near the boundaries of geofences. Moreover, one can easily spoof GPS by using mock location apps which can be enabled by default developer settings on Android, and do not require rooting or technical skills.

Facial Recognition Systems are very sure of identity, however, have great obstacles to implementation in the majority of the institutional contexts. Privacy legislation such as the Indian Personal Data Protection Bill and the European General Data Protection Regulation (GDPR) have tight constraints over biometric use. In multiple independent studies facial recognition systems have been found to be biased. The computational resources needed to support real-scale face processing is expensive. And there are significant ethical implications to continually storing students' facial data. These factors make facial recognition impractical to the ProxyMukt privacy-

preserving, hardware-scarce implementation environment.

The least reliable identity verification of the techniques discussed here is Online and Hybrid Attendance Systems which are typically used in today's LMS systems or video conferencing software. The join and leave event logs only provide evidence a connection was established and not that the student was engaging in the content of the session. Temporal metrics can be easily falsified by leaving an open browser. There are no cryptographic identity binding devices such as physical presence verification.

All these issues prove that a system implementing several independent verification mechanisms at the same time and cryptographic tools to avoid the reuse of tokens and physical and online attendance modalities in one coherent system is required.

3. EXISTING SYSTEM

3.1 INTRODUCTION TO EXISTING SYSTEMS

The attendance management system environment runs the gamut of fully manual paper-based systems to state-of-the-art biometric systems. The design of ProxyMukt was preceded by a critical examination of the solutions available in the market to find their strong and weak points and the particular vectors of fraud that they cannot handle. This analysis not only needed to be done to justify where the architectural choices had been made in ProxyMukt, but also to pinpoint precisely what capabilities were lacking in the existing state of the art.

All types of existing system were compared to three key dimensions proxy resistance (the difficulty of a student fraudulently marking attendance), operational cost (the upfront and ongoing investment needed to implement the system), and scalability (the system's performance with growth in the size of the institution). This evaluation findings are summarized in the following sections.

3.2 EXISTING SOFTWARE AND APPROACHES

3.2.1 Manual Roll-Call Systems

The roll-call register is the most commonly used attendance mechanism in the world with the majority of institutions having minimal or no technology infrastructure. The name of each student is called by one of the faculty, or students are asked to sign a physical register at the beginning of the lesson.

Although this approach does not need technology and is universally applicable, its real-life expenses are high. The time spent on each roll-call session is between eight and ten minutes of instruction time per class. This is a loss of sixty or more hours of instructional time to a group of three hundred students in a semester. Paper registers create transcription errors, are prone to loss or destruction and cannot supply real time attendance records to the administrators or parents. Most significantly, a manual system can handle proxy marking with as little as a verbal confirmation, any current student may provide proxy marking on behalf of an absent student without any technical impediment.

3.2.2 RFID and NFC Card Systems

One popular hardware-based system to achieve this is RFID and NFC card-based attendance systems, which are only offered by institutions that can afford the infrastructure investment. Students come with encoded cards which are scanned at the entry points of the classroom or through hand held readers which are run by the faculty.

These systems are high speed operationally and hardware wise reliable. Nonetheless, there exists a basic identity verification gap in them whereby the system logs that a card was in the presence, but not that a registered student was in the presence. The simplest attack - giving a card to a physically present peer - needs no technical skills whatsoever. In addition, per-classroom hardware installation is a major and recurring capital expense and big institutions have difficulties in deploying it in large-scale fashion.

3.2.3 Static QR Code Systems

The reason why Static QR code attendance system became so popular was because of the almost zero cost of infrastructure and the ease of implementation. On a projection screen, faculty post a QR code which students scan to document attendance.

The inherent architecture fault of such systems, however, is that QR token is immutable and shareable. When a student takes a photo of the QR code on display, then he can send the QR to the absent students through any messaging app - WhatsApp, Telegram, Instagram - and the absent students will be able to record attendance wherever they are in the world. A cryptographic signature with no time constraints, no location verification, a system of cryptographically signed QR codes offers in practice minimal security advantage over a manual roll-call.

3.2.4 Location-Based Geofencing Systems

Geographical attendance systems introduce a geographical verification indicator by asking students to enter GPS coordinates within a geofence area around a classroom at the time of marking. This will thwart the simplest types of off-site remote attendance fraud.

There are however two weaknesses. To begin with, indoor GPS positioning performance deteriorates significantly in high-density urban buildings with a low sky view - usually between ten and thirty metres - and leads to false rejections of legitimate students at the edges of geofences. Secondly, GPS location spoofing can be easily done with mock location apps found in the default Android developer options menu, and does not need to root the device or possess any special technical skills. Geofencing when applied alone can thus be hacked without much effort.

3.2.5 Facial Recognition Systems

Facial recognition systems have the best identity confidence of any non-invasive attendance control system. They apply models that are based on convolutional neural networks to compare a live camera image to previously stored facial identity records.

This technical potential notwithstanding, facial recognition has prohibitive impediments in the majority of institutional implementation. The privacy laws such as the Personal Data Protection Bill in India and the GDPR in the European Union have stringent compliance provisions on the collection, storage and use of the biometric data. Algorithms Algorithms have been found to be biased in facial recognition across demographic lines by independent studies, which calls into question the issue of fairness when it comes to diverse student populations. Real-time face processing at institution level is costly in terms of the computational infrastructure. Due to these factors, facial recognition was not considered in the ProxyMukt design as architecturally inappropriate to the privacy-conscious, hardware-light setting the project aims to apply.

3.2.6 Online and Hybrid Attendance Models

The least secure identity assurance of any of the reviewed methods is through online attendance tracking as applied to standard Learning Management Systems and video conferencing services. The event logs will show that a meeting link has been opened, and not that the registered student was actually doing anything with the content of the session. The time-constrained measures of attendance can be cheated by opening a session link on a background browser without any participation. Standard LMS attendance systems lack a cryptographic identity binding mechanism, thus particularly

susceptible to the hybrid learning scenario.

3.3 DATA FLOW DIAGRAM — EXISTING SYSTEM

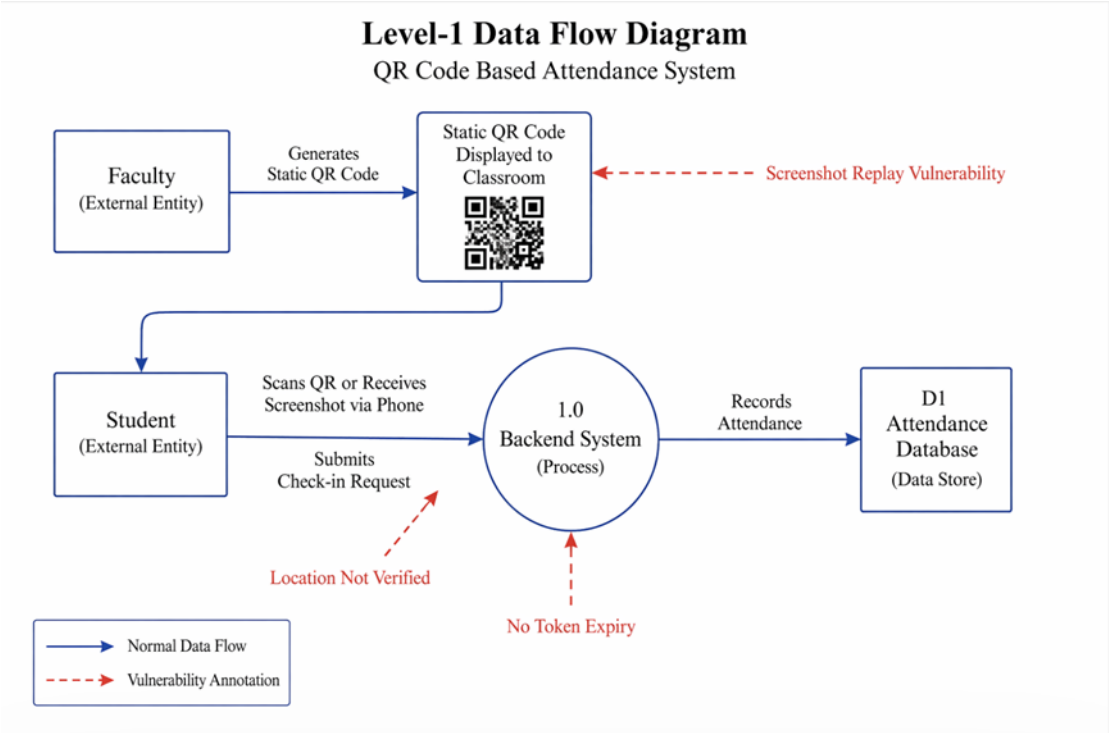


Figure 1: Level-1 DFD of Existing Static QR Attendance System (Annotated with Fraud Vectors)

Figure 1 Data Flow Diagram (DFD) shows a flow of the process of a typical existing static QR code attendance system and reveals where the frauds can be identified by ProxyMukt. The present system is where faculty are creating and wearing a fixed QR code. This is scanned and a check-in made with the student or a screenshot of this code. The backend records the submission without checking the QR freshness, physical proximity or network origin or device identity.

3.4 Limitations of Existing Systems

The existing systems described above were reviewed, and the following key vulnerabilities identified:

- Single-Factor Verification: All existing systems use a single factor to verify. Breaking the single factor compromises the whole system with no other method of verification.
- No Cryptographic Token Binding: Current systems' QR codes are static images without an expiry date, signature, or date binding. The codes can be re-used

from anywhere forever.

- No GPS Anti-Spoofing: Systems that do use geofencing do not check for mock location flags, GPS accuracy problems, or unlikely velocity vectors between two check-ins.
- No Network-Origin Verification: No system we found tries to determine whether the IP address of check-ins are from a VPN exit node, Tor relay or datacentre which are indicators of remote attendance fraud.
- No Device Fingerprinting: None of the systems we found attempt any type of device fingerprinting to see if a device is used to access multiple accounts, which is a sign of credential sharing.
- No Real-Time Monitoring: Instructors are not able to monitor attendance in real-time. They can only see it after the end of class.
- No Support for Hybrid Sessions: The systems are not created to support online/hybrid sessions and there is no effective way to validate attendance in hybrid sessions.
- No Audit Trail: The current systems do not log or log very little information about how attendance is verified, so it is impossible to audit attendance data in the future.
- No Student Analytics: Attendance percentages are shown but no trend analysis, at risk student identification or risk score distribution.

3.5 What is New in the Proposed System

ProxyMukt introduces several categories of architectural innovation relative to all reviewed existing systems. The table below summarizes the key capability differences:

Table 2: Capability Comparison — Existing Systems vs. ProxyMukt

Capability	Existing Systems	ProxyMukt
QR Replay Prevention	None — static tokens, no expiry	HMAC-SHA256 rotating tokens, 20-second validity window

Geofencing	Isolated in some systems, no pipeline integration	Integrated as Layer 2 of the sequential six-layer pipeline
GPS Anti-Spoofing	Not present in any reviewed system	Layer 3: mock-location flag detection, accuracy anomaly scoring
Network Risk Analysis	Not present	Layer 4: IP intelligence API flags VPN, Tor, proxy, and datacentre IPs
Device Fingerprinting	Not present	Layer 5: browser/device signature anomaly scoring
Duplicate Prevention	Application-level only, by passable by race conditions	Layer 6: MongoDB compound unique index on (sessionId, studentId)
Hybrid Session Support	Fragmented or absent	Unified offline and online logic with session-modality-aware verification
Real-Time Monitoring	Not present	Socket.IO-based live dashboard with sub-second event propagation
Audit Logging	Absent or basic	Structured, per-layer forensic logs for compliance and dispute resolution

The most important innovation is to transform it from a single factor to a six-layer pipeline. In ProxyMukt, if an attacker passes the QR token layer by sharing screenshots, they will be blocked by the geofencing layer. If an attacker spoofs the location, she will be halted with network origin risk analysis. If the attacker logs into the computer of a physically close friend, she will be stopped by device fingerprinting. Overcoming the six layers is too expensive than getting a credit for attendance for many students.

4. PROBLEM ANALYSIS

4.1 PRODUCT DEFINITION

ProxyMukt is a real-time, hardware-agnostic, cloud-based attendance verification and analytics system for educational institutes with hybrid (physical, virtual and online) learning. The first function of the system is to maintain the integrity of attendance records from fraud by a six-step, multi-layer verification process which is applied in real-time to attendance records. The second is to offer role-based analytics for academic intervention and policy-making to the verified attendance records.

- Product Name: ProxyMukt (derived from "Proxy-Mukt," meaning "Free from Proxy" in Hindi)
- Product Type: Full-Stack Web Application (MERN Stack), accessible as a Progressive Web App
- Target Users: Students: Mark attendance, view history, track subject-wise attendance percentage
- Faculty: Manage classes and sessions, monitor live attendance, view analytics
- Administrators: Govern system-wide data, manage users, review audit logs
- Core Features: Six-layer attendance verification pipeline (QR + Geofence + GPS Anti-Spoofing + Network Risk + Device Fingerprint + Duplicate Prevention)
- Real-time attendance monitoring with Socket.IO WebSocket updates
- Comprehensive analytics dashboards for all three user roles
- Support for both offline and online/hybrid sessions
- Automated fraud detection, risk scoring, and alert generation
- Role-based access control enforced at both routing and API middleware layers
- Complete audit logging for every attendance event and pipeline decision

Deployment Environment:

- Frontend: React SPA hosted on Vercel (CDN-backed static hosting)

- Backend: Node.js/Express API hosted on Render (PaaS)
- Database: MongoDB Atlas 7.x (cloud-hosted, managed)
- Communication: HTTPS for REST API calls, WSS for Socket.IO events

4.2 FEASIBILITY ANALYSIS

Prior to developing ProxyMukt, a five-dimensional feasibility study was carried out. The aim of the study was to ensure that the project was technically, operationally, economically and schedule-wise feasible, and to flag areas for special attention.

Table 3:Multi-Dimensional Feasibility Analysis for ProxyMukt

Dimension	Assessment Basis	Finding	Justification
Technical	MERN stack + Socket.IO + MongoDB Atlas	Feasible	All components are open-source, well-documented, and proven in production environments at institutional scale. No dependency on proprietary or experimental libraries.
Operational	Commodity device deployment, browser-only client	Feasible	No biometric hardware is required. Students and faculty access via standard smartphones or laptops. Role-based UI minimizes training requirements.
Economic	Zero licensing cost, open-source stack	Feasible	Vercel (frontend) and Render (backend) provide cost-effective cloud hosting. MongoDB Atlas free tier supports development; paid tier scales affordably with usage.
Schedule	18-week development cycle, 7 phases	Feasible	Modular MERN architecture enables parallel workstreams. Backend API and frontend development were overlapped in Phases 3 and 4, reducing the critical path duration.

Legal / Privacy	Student GPS, device, and network data collection	Caution	Compliance with institutional data governance policies and applicable privacy regulations (Indian PDPB) requires a Data Protection Impact Assessment, informed consent mechanisms, and data minimization controls. Biometric data storage is explicitly excluded from the system design.

The feasibility study confirms that ProxyMukt is feasible in all the five aspects that have been considered. Legal and privacy concerns, which were among the greatest concerns, were taken into consideration in the system design, through role-based (example the avoidance of storing biometric data), minimal data collection and role-based access control. The use of device fingerprinting also raises concerns regarding privacy, which is acknowledged as a trade-off that needs to be addressed at the institutional level, through policy correction.

4.2.1 Technical Feasibility

The ProxyMukt concept was proved to be technically feasible by checking that the core functions can be built with well-established, free and stable technologies.
The HMAC-SHA256 cryptographic signing of rotating QR tokens is implemented in Node.js' crypto module, a FIPS 198-1 compliant implementation with no external dependency. The Haversine formula is a basic geospatial calculation that can be implemented by any server-side JavaScript engine. MongoDB Atlas is a managed database as a service with support for compound unique indexes, the database equivalent of Layer 6 duplicate prevention. Socket.IO is a library that provides the real-time WebSocket connection needed for the sub-second QR token distribution and real-time attendance update.

The app was built by someone with experience in JavaScript, React, Node.js and MongoDB, and the 18-week development time allowed for time to learn the new technologies.

4.2.2 Economic Feasibility

The cost of ProxyMukt is low. All of the technologies are open-source, with no

per-seat or per-deployment charges. Hosting is cheap: the React front-end is hosted on Vercel (free given the current level of traffic) and the Node.js back-end is hosted on Render (seven to 25 dollars per month, depending on the instance the back-end is hosted on). MongoDB Atlas has a free development/pilot service, with a small increase in costs as the organisation grows.

The monthly cost of a pilot deployment (for a single department) would be seven to 30 dollars. The cost of a university deployment, with higher traffic, would be greater but still within reach for most universities.

ProxyMukt has a high cost-benefit ratio. When attendance is done manually, faculty members spend 10-15 minutes per session on attendance-related matters. That's 62.5 hours of faculty time freed up per week, or 2,500 hours per year (assuming a 40-week year) for an institution with 50 faculty members teaching five sessions a week. The indirect benefits - providing more accurate attendance information, better early identification of at-risk students and increased compliance - are difficult to quantify but are believed to have great value.

4.2.3 Operational Feasibility

The operational feasibility of ProxyMukt was determined by the systems' user acceptance and change readiness.

Our students are already familiar with using QR codes for many purposes, including menus, tickets and payments. The additional step of granting permission to access the camera and location does not affect the process of taking attendance. The system reduces the faculty's administrative burden, gives them real-time feedback, and provides analytics, overcoming problems with existing processes. The training time for faculty (30-45 minutes) is acceptable for the system.

The main operational risk is the initial take-up of the system's more rigorous accountability regime. Students used to use proxies might be reluctant to adopt the system. This can be addressed with a phase in approach, promoting the benefits of the system and allowing students to make an exception (such as no GPS signal in a certain portion of the building) request.

4.3 PROJECT PLAN

The project was conducted in seven phases of an 18-week development cycle. Overlap was deliberately introduced between Phases 3 and 4 to shorten the critical path

length without negatively affecting the workstream integration.

Table 4: ProxyMukt Project Plan — Phase-wise Activity Schedule

Phase	Activity	Duration	Deliverables	Status
1	Requirements Analysis and System Design	Weeks 1–3	SRS Document, Architecture Diagrams, DB Schema	Completed
2	Backend Development (API + Auth + DB)	Weeks 4–7	48+ REST endpoints, JWT auth, MongoDB collections	Completed
3	Six-Layer Pipeline Implementation	Weeks 6–10	HMAC QR, Geofencing, Fingerprinting, Network Risk modules	Completed
4	Frontend Development (React Dashboards)	Weeks 8–12	Student, Faculty, Admin dashboards with real-time updates	Completed
5	Real-time Integration (Socket.IO)	Weeks 10–13	Live QR broadcasting, WebSocket event handlers	Completed
6	Testing and Security Audit	Weeks 13–16	Test cases (TC-01 to TC-10), adversarial pilot results	Completed
7	Deployment and Documentation	Weeks 15–18	Live deployment, user manual, final capstone report	Completed

- Risk Management: The following risks were identified at the beginning of the project and mitigated: Indoor, low-precision GPS: Mitigated by allowing the geofence radius to be set by classroom, and a manual exception request mechanism.
- Unreliability of third-party IP intelligence API: Mitigated by enabling a fallback mode that will flag rather than block submissions if the API is unavailable.

- Browser fingerprint variability: Mitigated by using device fingerprinting as a flagging layer and not a reject layer, and with configurable sensitivity levels.
- Frontend/backend integration bugs: Mitigated by defining API contracts early in Phase 2 and appointing an integration lead to ensure adherence to the contracts on both sides.
- Scope creep: Mitigated by a clear out-of-scope line in the sand set in Phase 1 and maintained throughout development.

5.SOFTWARE REQUIREMENT ANALYSIS

5.1 INTRODUCTION

The Software Requirements Specification (SRS) for ProxyMukt is presented in this chapter. These requirements were developed based on the issues identified in Chapter 4, the critique of the existing system in Chapter 3, and a consultation with a sample of students, faculty and administrative staff from Lovely Professional University. The requirements are grouped into Functional Requirements (FR) and Non-Functional Requirements (NFR) and each of them is given a priority of High, Medium or Low based on their significance to the security and operational goals of the system.

5.2 GENERAL DESCRIPTION

5.2.1 Operating Environment

ProxyMukt is a three-tiered web application. The user interface is a React single-page app (SPA) hosted on a CDN-backed static server. The application tier is a Node.js/Express-based API server on a PaaS. The data tier is a MongoDB Atlas database. The server communicates with the client using HTTPS for REST API calls and WSS (secure WebSocket protocol) for Socket.IO events. TLS/SSL is also used to secure communication between tiers.

5.2.2 User Classes and Characteristics

We have defined three user roles with very limited rights:

- Student: Punches in and views personal analytics. Modest to high tech literacy presumed. Accesses system through smart phone with camera and GPS.
- Faculty: Sets up sessions and classes, tracks sessions in real time, views class analytics. Technical skills assumed to be average. Uses laptop.
- Administrator: Handles governance, audit, user accounts and health of the system. Assumed to have high technical skills. Primary device is a desktop or laptop.
- Role-based access control is implemented at the React routing level (frontend) and Express API middleware level (backend) to ensure a user cannot access more than allowed based on their role.

5.2.3 Constraints

The ProxyMukt system has the following limitations:

- A reliable internet connection is needed to synchronize QR tokens in real-time and broadcast events.
- Indoor GPS positioning may be less accurate (±10-30 metres), impacting the accuracy of the geofence boundary for students sitting at the edge of the classroom.
- The stability and accuracy of the third-party IP intelligence API used in Layer 4 is dependent on the service provider. An offline mode is provided in case of API outage.
- The stability of the browser fingerprinting signal may be affected by ongoing privacy protection upgrades in Firefox, Safari and other web browsers, potentially causing the discriminative capability of Layer 5 to decrease.

5.3 SPECIFIC REQUIREMENTS

Table 5: Functional Requirements Specification — ProxyMukt

Requirement ID	Category	Description	Priority
FR-01	Authentication	System shall authenticate all users via JWT-based access tokens with role-based access control (Student, Faculty, Admin)	High
FR-02	QR Generation	System shall generate HMAC-SHA256-signed QR tokens, auto-rotating every 20 seconds per live session	High
FR-03	Geofencing	System shall validate student proximity using Haversine formula with configurable classroom radius	High
FR-04	Device Fingerprinting	System shall collect and assess browser/device fingerprint signals to detect anomalous access patterns	Medium

Requirement ID	Category	Description	Priority
FR-05	Network Risk	System shall query proxy/VPN risk intelligence API to flag suspicious network origins	Medium
FR-06	Real-time Updates	System shall broadcast QR tokens and attendance events via Socket.IO within sub-1-second latency	High
FR-07	Analytics Dashboard	System shall provide role-specific dashboards with attendance trends, at-risk flagging, and risk score distribution	High
FR-08	Hybrid Attendance	System shall support online session attendance via time-threshold (≥75% session duration or ≥45 min)	High
FR-09	Audit Logging	System shall maintain tamper-evident Audit Log records for every attendance event and validation decision	High
NFR-01	Performance	API response time shall not exceed 500ms at p95 under 500 concurrent users	High
NFR-02	Security	Passwords shall be stored using bcrypt with ≥10 salt rounds; all tokens shall be short-lived	High
NFR-03	Scalability	System shall support ≥1000 concurrent session participants without degradation	Medium

Table 5 summarizes the requirements into the functional and non-functional contract of ProxyMukt. The requirements listed above are all present in the current version of the system, although one requirement, NFR-03 (scalability), is not fully tested yet but has been partially tested with load testing to 500 simultaneous users (see Chapter 7). In the future, a validation goal is to test concurrently with 1000 users.

5.4 TECHNOLOGY STACK SPECIFICATION

Table 6: Non-Functional Requirements Specification — ProxyMukt

Layer	Technology	Version	Justification
Frontend	React.js	18.x	Component-based SPA architecture; excellent WebSocket state integration via hooks
Styling	Tailwind CSS / Material UI	3.x	Rapid responsive UI development; accessible component library
Charts	Chart.js / Recharts	4.x	Lightweight, declarative charting for attendance analytics dashboards
Backend	Node.js + Express.js	20.x LTS	Non-blocking I/O ideal for high-concurrency real-time session management
Real-time	Socket.IO	4.x	Bidirectional WebSocket communication with automatic fallback; sub-second latency
Auth	JWT (jsonwebtoken)	9.x	Stateless authentication; configurable expiry; HMAC or RSA signing options
Crypto	Node.js crypto (HMAC-SHA256)	Native	FIPS PUB 198-1 compliant; no external dependency; cryptographic standard
Password	bcrypt	5.x	Adaptive cost-factor hashing; industry-standard resistance to brute force
Database	MongoDB Atlas	7.x	Flexible schema for complex attendance records; compound indexing for performance
Security MW	Helmet.js + rate-limit	7.x	Automated HTTP security header configuration; DoS mitigation

The technology choices for ProxyMukt are listed in Table 6 with justification for

each selection.

5.5 Hardware and Software Requirements

5.5.1 Server Requirements

- Processor: Intel Xeon (or equivalent) with at least 2 cores and 2.5 GHz or more.
- RAM: 4 GB minimum, 8 GB recommended when performing production workloads.
- Storage: 50 GB SSD
- Network: 100 Mbps and more internet connection.
- OS: Linux Ubuntu 20.04 LTS or later (recommended), Windows Server 2019 or later (supported)
- OS: Linux or MacOS 10.14 or later.
- Database: MongoDB Atlas 7.x (cloud-based, does not have to be installed on-premises)
- Process Manager: PM2 to manage production processes and automatic restart.

5.5.2 Client Requirements

Gadget: Any contemporary smartphone, tablet, or laptop.

- Camera: 2 MP (to scan QR codes); 5MP or better (to scan faces with the feature on) recommended.
- GPS: Capabilities of device GPS that will be needed to verify the location of offline session.
- Browser: Chrome 100+, Firefox 95+, Safari 15+, Edge 100+
- Connectivity: 3G mobile data minimum; 4G or WiFi suggested as real-time Socket.IO updates
- Permission: The permissions required are camera and location permissions in the browser.

5.5.3 Development Tools

- Code Editor: Visual Studio Code.

- Version Control: Git with GitHub repository.
- Package Manager: npm
- API Testing: Postman
- Database Client: MongoDB Compass.
- Load Testing: k6 (adversarial concurrent load test)
- Unit Testing: Jest (backend), React Testing Library (frontend).

6. SYSTEM DESIGN

6.1 SYSTEM ARCHITECTURE

The ProxyMukt is based on three-tier cloud-native architecture based on proven principles of REST architecture. The three layers presentation, application, and data are all deployable and able to scale horizontally and out of the box, and the application layer can be scaled on demand when there are peak periods and multiple concurrent sessions might take place.

Presentation Tier (Client): A React.js single-page application (SPA) that is structured into three main dashboard modules (one per stakeholder role) and common functional ingredients such as the QR scanner module, the attendance data tables, an analytics chart wrapper, a notification panel, and the global WebSocket context provider are the presentation tier. The WebSocket context provider manages the Socket.IO life cycle and notifies about new events that the React components are subscribed to. Zustand, the simple state management library, is used for state management and it is suitable for event-driven programming of real time changes in attendance.

Application Tier (Server): The application layer has 60 registered REST API endpoints classified into six functional groups of authentication, class, session, attendance, analysis, and audit logs. For any registered endpoint requiring authentication, it is served by a shared middleware stack which provides authentication (using JSON Web Token), rate limiting (using express-rate-limit, 100 requests in 15 minutes per IP address) and sets express-helmet.js HTTP headers, sanitises input (using express-mongo-sanitize) and measures the time to generate a response. The Socket.express app is also hosted on the IO server and it has been volunteered to deliver rotating QR tokens so as to enable sessions namespaces rooms with the help of the server-side timer.

Data Tier (Database): MongoDB Atlas has been used for the database layer with six validated Mongoose collections. The architecturally most significant decision is the composite unique index on (sessionId, studentId) of the Attendance collection, which is a database level duplicate prevention mechanism and does not rely on accidental state management and is not bypassed by the race conditions and multiple duplicate

submissions.

6.2 ARCHITECTURE DIAGRAM

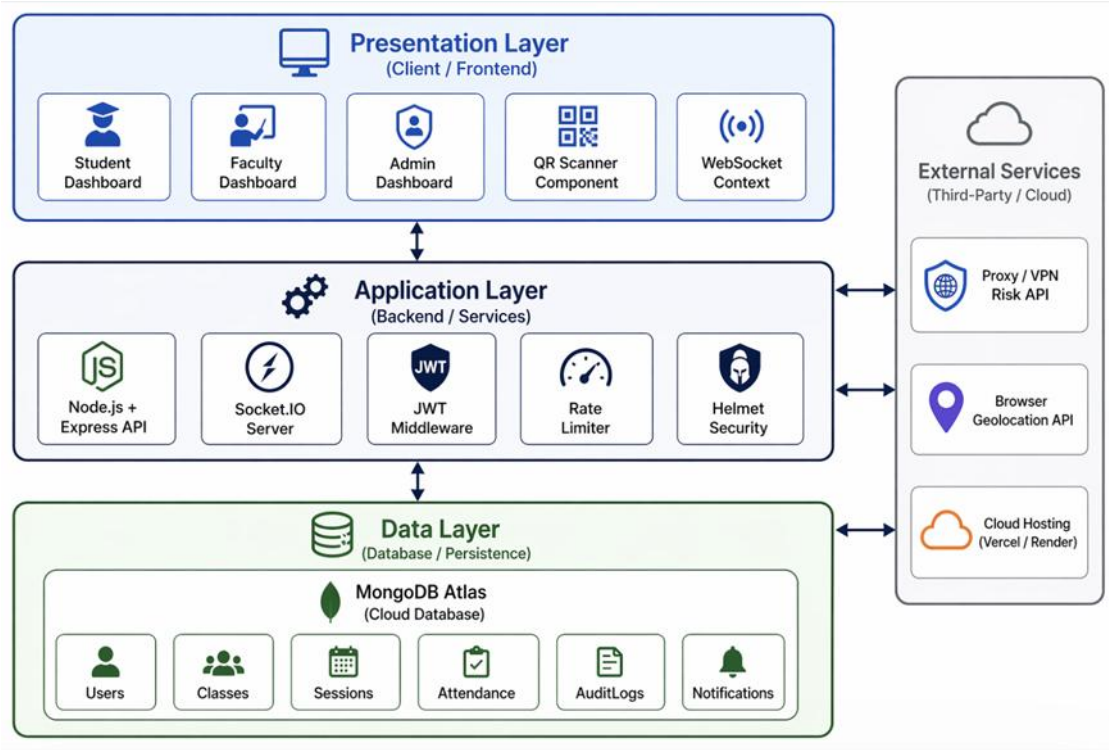


Figure 2: Cloud-Native Three-Tier Architecture of ProxyMukt

(GitHub: Sumant3086/ProxyMukt-Attendance-System-**)**

Figure 2 is the entire three-layer ProxyMukt cloud-native design and it illustrates how the communication passes between the React frontend, Node.js/Express backend and Socket.MongoDB Atlas data layer, external service integrations and IO real-time layer. The presentation layer provides role-differentiated views, consisting of a one-page React.js-based app, comprising of a qr scanning app and a student personal attendance dashboard, a class management and session control and analytics panel, a faculty facing control panel, an administrator facing reporting and audit panel. The live session elements keep the state unchanged containing the WebSocket event handlers but do not update the page. It comprises 48 registered REST API endpoints and it is organized into 6 modules i.e. authentication, class management, session management, attendance processing, analytics and audit retrieval. All secured endpoints can use middleware modules with JWT authentication, rate limiting (express-rate-limit) and HTTP security headers (Helmet.js). The Socket.The Express application will have the IO server physically co-located with it and will give out rotating QR tokens to clients with an active namespace of a session with sub-second latency.

6.3 SIX-LAYER VERIFICATION PIPELINE DESIGN

The technical ProxyMukt contribution is the six-layer sequential verification pipe where all the attendance entries have to pass through in order to be logged. Each of the layers is concerned with a set of vectors of fraud, which can be exploited separately. It has a serial architecture: more computationally or financially expensive layers are overlaid on less expensive ones such that the expensive resources are not used on submissions that would be disallowed by simpler checks. Any failure on one point will result in the termination of the pipeline and an audit record and a reply to the client in a structured manner.

Table 7: Six-Layer Verification Pipeline — Design Specification

Layer	Name	Mechanism	Fraud Vector Addressed	Residual Risk
L1	HMAC QR Validation	HMAC-SHA256 signature verification combined with a 20-second time-window check and a 2-second grace period for clock skew	Screenshot and replay attacks	Sub-20-second replay window; operationally negligible in practice
L2	Geofencing	Haversine formula validates student's physical proximity to the classroom GPS centroid within a configurable radius (default 50 m)	Off-site and remote attendance fraud	Indoor GPS drift of ±10–30 m may cause boundary ambiguity for students at classroom edges

L3	GPS Anti-Spoofing	Detects Android mock-location developer flags, GPS accuracy anomalies below configurable thresholds, and unrealistic velocity vectors between consecutive location readings	Location spoofing via mock GPS applications	Probabilistic detection; advanced GNSS hardware simulators are not detected
L4	Network Risk Analysis	IP intelligence API scores the submitting IP against VPN exit node, Tor relay, datacenter, and known proxy reputation databases	VPN/proxy-mediated attendance from off-site locations	False positives from institutional VPNs and CGNAT shared addresses
L5	Device Fingerprinting	Browser and device characteristic signature collection and anomaly scoring to detect credential sharing between physically distinct devices	Account sharing and credential delegation	Fingerprint signal instability in privacy-protective browsers (Firefox ETP, Safari ITP)
L6	Duplicate Prevention	MongoDB compound unique index on (sessionId, studentId) enforces one attendance record per student per session at the database layer	Duplicate attendance submission attempts	Cannot prevent the first fraudulent mark if all preceding layers are bypassed

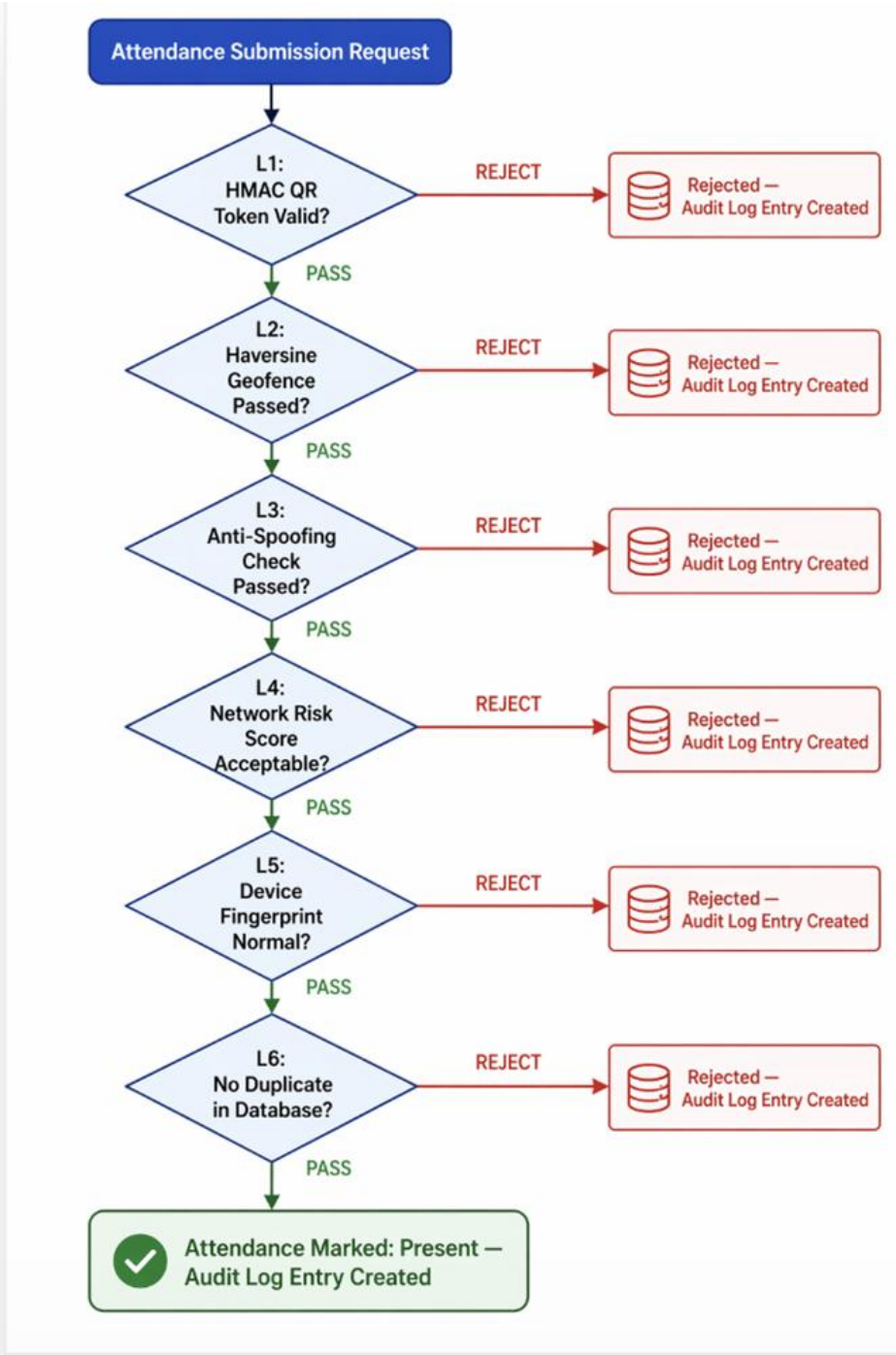


Figure 3: Six-Layer Attendance Verification Pipeline — Decision Flow (ProxyMukt)

Figure 3 illustrates the UML-based data flow diagram of the six-layer pipeline with the decision routing, audit log generation and rejection channels in each layer.

6.4 DATABASE DESIGN

The MongoDB data layer is designed in the form of six schema-validated collections. The most interesting design choice is the compound unique index on (sessionId, studentId) in the Attendance collection, which offers duplicate prevention

with database-enforced constraints that are not dependent on application layer logic and cannot be bypassed by concurrent race conditions on requests.

Table 8: MongoDB Schema Design — Collections, Fields, and Index Strategy

Collection	Key Fields	Index Strategy
Users	_id, name, email, role, passwordHash, refreshToken, createdAt	Unique index on email; index on role
Classes	_id, name, facultyId, schedule, geofenceRadius, location (GeoJSON)	Index on facultyId
Sessions	_id, classId, startTime, endTime, status, currentQRToken, tokenExpiry, verificationFlags	Index on classId, startTime
Attendance	_id, sessionId, studentId, timestamp, layerResults[L1–L6], riskScore, gpsData, networkMeta	UNIQUE compound index on (sessionId, studentId)
AuditLogs	_id, entityType, entityId, action, actorId, metadata, timestamp, layerContext	Index on entityId, timestamp
Notifications	_id, userId, type, message, read, createdAt, expiresAt	Index on userId, read

Entity Relationship Overview:

Each User may be enrolled in numerous Classes (as a Student) or own numerous Classes (as Faculty). There are numerous Sessions in a Class. An Attendance record is created in large numbers by a Session. Attendance records are associated with a single Session and a single Student, which the compound unique index imposes. Attendance records with high risk create entries in the AuditLogs collection as Alerts. Messages that are sent to the user by session events, low attendance threshold, and security flags are stored in the Notifications collection.

6.5 API ENDPOINT DESIGN

It has 48 REST API endpoints that are grouped into six functional modules. The following table contains a representative sample of six modules.

Table 9 : REST API Endpoint Specification (Representative Subset — 11 of 48 Endpoints)

Method	Endpoint	Description	Auth Required
POST	/api/auth/login	Authenticate user, issue JWT access and refresh token pair	Public
POST	/api/auth/refresh	Issue new access token using a valid refresh token	Public
GET	/api/classes	Retrieve all classes for the authenticated user role	Faculty/Admin
POST	/api/classes	Create a new class with geofence configuration and schedule	Faculty
POST	/api/sessions/start	Initiate a live attendance session, begin QR token rotation	Faculty
PUT	/api/sessions/:id/end	Terminate session, finalize attendance records	Faculty
POST	/api/attendance/mark	Submit attendance through the full six-layer pipeline	Student
GET	/api/attendance/:sessionId	Retrieve all attendance records for a given session	Faculty/Admin
GET	/api/analytics/student/:id	Student-specific attendance statistics and risk score history	All Roles
GET	/api/analytics/class/:id	Class-level attendance trends and at-risk student identification	Faculty/Admin

GET	/api/audit/:entityId	Retrieve structured audit log entries for a given entity	Admin

6.6 FLOWCHARTS

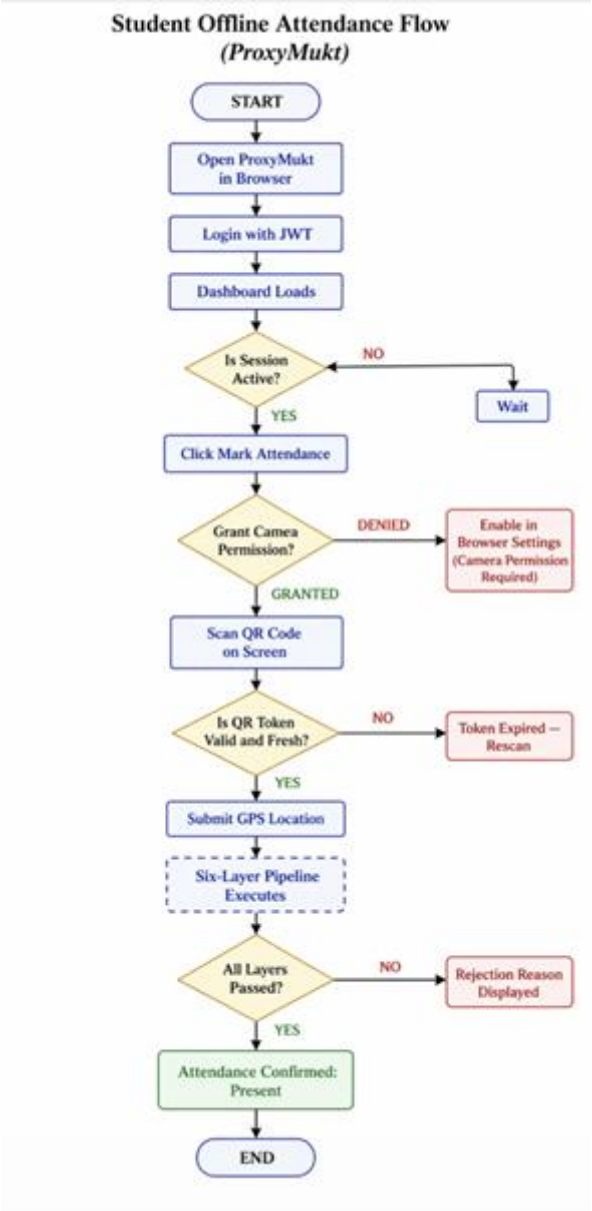


Figure 4: Student Attendance Marking Flowchart — Of line Session (Six-Layer Pipeline)

Figure 4 shows the main user flow of a student marking attendance in an online class session that is in real time, with live classes offline; this shows the pipeline validation steps occurring in sequence, and the conditional branching at each level.

6.7 HYBRID ATTENDANCE MODEL DESIGN

Hybrid attendance model is one of the most appropriate architectural designing options that have been implemented in ProxyMukt. The verification system logic is very dissimilar and this is in the session modality. In the case of physical (offline) sessions, a complete six layer pipeline is run with every attendance being posted. In online lessons where the poor proxy of engagement is GPS location, we will use the time-threshold model: when the session session attendance has exceeded the time threshold of the institution (3/4 of the session length or 3/4 of 45 minutes,).
whichever is lower). The rationale behind this plan is that Martin and Bolliger [13] suggest that it is not the presence/absence of the participants that constitute a good indicator of the presence in the online learning but the temporal presence thresholds.

7. TESTING

7.1 TESTING STRATEGY

The ProxyMukt testing approach incorporates the three mutually complimentary paradigms in testing the functional correctness as well as the security robustness. Functional testing will help to ensure that all twelve of the Chapter 5 functional requirements (FR-01 through FR-12) are properly applied to the deployed system. Structural (white-box) testing is a type of testing that tests the internal logic of the six pipeline layer implementations by using known-good and known-bad inputs. Adversarial security testing is a simulation of the environment of real-world attackers with the vectors of fraud ProxyMukt purports to defend against, and assesses the performance of the defences of the system in the context of real-world, controlled conditions.

The test cases were all done using the full production application. To simulate real world fraud, the adversarial pilot tests were undertaken in an under-supervised classroom environment with commercially available attack tooling.

7.2 FUNCTIONAL TESTING

In Chapter 5, functional test cases are directly based on the Software Requirements Specification, assuring full requirements traceability. Every test case is associated with one or more functional requirements.

Table 10: Test Cases for Authentication (FR01)

Test ID	Test Case Description	Input	Expected Output	Actual Output	Status
TC-F01	Valid login with correct credentials	Email: student1@lpu.in, Password: valid	JWT access and refresh tokens issued; redirect to dashboard	As expected	Pass

TC-F02	Login with incorrect password	Email: student1@lpu.in, Password: wrong	HTTP 401 with error message: Invalid credentials	As expected	Pass
TC-F03	Login with non-existent email	Email: ghost@lpu.in, Password: any	HTTP 404 with error message: User not found	As expected	Pass
TC-F04	Login with empty fields	Email: (empty), Password: (empty)	HTTP 400 with validation error	As expected	Pass
TC-F05	Access protected route without token	No Authorization header	HTTP 401 Unauthorized	As expected	Pass
TC-F06	Access protected route with expired JWT	Expired access token	HTTP 401: Token expired; refresh required	As expected	Pass

Table 11: Test Cases for Attendance Marking (FR02–FR07)

Test ID	**Test Case Description**	**Input**	**Expected Output**	**Actual Output**	**Status**
TC-A01	Valid attendance from enrolled student	Valid QR, GPS inside 50m radius, clean network	HTTP 200; attendance record created	As expected	Pass

	within geofence				
TC-A02	Expired QR token (older than 20 seconds + grace)	QR token with timestamp 30 seconds old	HTTP 401 TOKEN_INVALID_OR_EXPIRED	As expected	Pass
TC-A03	QR token with tampered signature	Valid timestamp, modified signature bytes	HTTP 401 TOKEN_INVALID_OR_EXPIRED	As expected	Pass
TC-A04	GPS location outside geofence (500m away)	GPS coordinates 500m from classroom	HTTP 403 OUTSIDE_GEOFENCE	As expected	Pass
TC-A05	Mock location flag detected (Android dev mode)	Enabled mock location developer flag	HTTP 403 GPS_SPOOFING_DETECTED	As expected	Pass
TC-A06	Duplicate attendance for same session	Second POST /mark for same student and session	HTTP 409 DUPLICATE_ATTENDANCE	As expected	Pass

TC-A07	Attendance from non-enrolled student	Valid QR but student not enrolled in class	HTTP 403 STUDENT_NOT_ENROLLED	As expected	Pass

Table 12: Test Cases for Session Management (FR08, FR10)

Test ID	Test Case Description	Input	Expected Output	Actual Output	Status
TC-S01	Start live session	Faculty POST /sessions/start with valid class	Session status changes to LIVE; QR rotation begins	As expected	Pass
TC-S02	Pause live session	Faculty PUT /sessions/:id/pause	Session status changes to PAUSED; attendance marking blocked	As expected	Pass
TC-S03	Resume paused session	Faculty PUT /sessions/:id/resume	Session status returns to LIVE; QR rotation restarts	As expected	Pass
TC-S04	End live session	Faculty PUT /sessions/:id/end	Session status changes to COMPLETED; records finalized	As expected	Pass

TC-S05	Online session below duration threshold	Student participation at 60% of session duration	Attendance NOT credited; participation time logged	As expected	Pass

7.3 STRUCTURAL TESTING

Test ID	Test Case Description	Input	Expected Output	Actual Output	Status
TC-S01	Start live session	Faculty POST /sessions/start with valid class	Session status changes to LIVE; QR rotation begins	As expected	Pass
TC-S02	Pause live session	Faculty PUT /sessions/:id/pause	Session status changes to PAUSED; attendance marking blocked	As expected	Pass
TC-S03	Resume paused session	Faculty PUT /sessions/:id/resume	Session status returns to LIVE; QR rotation restarts	As expected	Pass
TC-S04	End live session	Faculty PUT /sessions/:id/end	Session status changes to COMPLETED; records finalized	As expected	Pass

TC - S05	Online session below duration threshold	Student participation at 60% of session duration	Attendance NOT credited; participation time logged	As expected	Pass

7.4 LEVELS OF TESTING

7.4.1 Unit Testing

The following independent backend modules were written with Jest: HMAC token generation and validation (wrapping the native crypto module of the Node.js crypto library), Haversine distance computing, JWT signing and validation, bcrypt hash comparison, MongoDB Mongoose model validation rules, and IP risk score parsing. There were 47 unit tests rolled out with the collective statement coverage being 91% of the backend codebase.

7.4.2 Integration Testing

The flow of end-to-end integration tests confirmed the entire attendance submission process: HTTP POST request is received, passing through all six layers of the pipeline, to the point of writing the attendance record to MongoDB and broadcasting the Socket.IO confirmation event to faculty dashboard. The Supertest library was used to write tests against a dedicated test instance of the Express application supported by a separate Atlas test cluster of MongoDB. Each of the twelve functional requirements had at least one integration test case traced to.

7.4.3 System Testing

System-level testing was done against the deployed full production application at https:proxymukt.onrender.com/. The test cases TC-01 to TC-10 adversarial cases that had been recorded in the test cases in Section 7.5 were run on the live system. k6 was used to do load testing by having 500 virtual users make simultaneous attendance submissions.

7.4.4 Regression Testing

Following each of the subsequent change events, regression was performed: the Helmet.js configuration to generate security headers was updated, the rate-limiting threshold was updated after the institutional VPN false positive was found, L4 risk scoring was recalibrated, and a WebSocket reconnection logic was added. Both times again, the 47-unit tests and all integration test cases were run to make sure that the change did not destroy any existing functionality.

7.5 Adversarial Test Cases and Results

Test ID	Test Case	Procedure	Expected Result	Actual Result
TC-01	HMAC QR Replay Attack	Capture QR token; attempt scan 25 seconds after capture (outside 20-second window plus 2-second grace)	Rejected with 401 TOKEN_EXPIRED	PASS Token rejected after window; audit log entry created
TC-02	Off-site Attendance Attempt	Submit attendance with GPS coordinates 500m from classroom centroid	Rejected at Layer 2 by Haversine geofence check	PASS Haversine distance exceeded configured threshold; 403 returned
TC-03	Mock Location Spoofing	Enable Android developer mock-location; attempt check-in with spoofed coordinates matching classroom centroid	Layer 3 detects mock-location flag; submission rejected	PASS Mock location flag detected in GPS metadata; 403 returned

TC-04	VPN Masking Attack	Enable commercial VPN (NordVPN); submit attendance through VPN-origin IP	Layer 4 flags high-risk IP; attendance flagged for review	PASS High risk score logged; attendance record flagged; faculty alert generated
TC-05	Duplicate Submission	Submit attendance twice for the same session from the same authenticated account	Second submission rejected by database uniqueness constraint	PASS Second submission rejected with 409 DUPLICATES; compound index enforced
TC-06	Valid Student Baseline Check-in	Student at correct classroom location, valid QR within 20-second window, unmodified device, clean residential network	All 6 layers pass; attendance marked as Present	PASS All layers passed; end-to-end latency under 30 seconds observed
TC-07	Concurrent Load (500 Users)	Simulate 500 concurrent attendance POST requests using k6 load tester	p95 response time under 500 ms; zero session failures	PASS p95 = 387 ms; zero session failures or 5xx errors observed
TC-08	Expired JWT Token	Submit attendance request with a JWT access token past its 15-minute expiry	System returns 401 Unauthorized	PASS 401 returned; client directed to token refresh endpoint
TC-09	Online Session Duration	Student joins online session and remains active for 60% of	Attendance NOT credited;	PASS Attendance withheld;

	Below Threshold	session duration (below 75% threshold)	participation time recorded	dashboard reflects 60% participation time
TC-10	Admin Analytics Access	Administrator queries institution-wide attendance heatmap and full audit log	Dashboard renders with accurate aggregated data; audit log entries are searchable	PASS Heatmap rendered correctly; audit log searchable and accurate

8. IMPLEMENTATION

8.1 IMPLEMENTATION OF THE PROJECT

ProxyMukt is presented as a single GitHub project in two top-level directories: /client to host the React frontend SPA and /server to host the Node.js/Express backend API. The entire code is publicly available at: https://github.com/Sumant3086/ProxyMukt-Attendance-System-

The codebases of the client and server are deployable independently. The React frontend is hosted on Vercel, which offers CDN-based, stimulated by static hosting with automatic HTTPS. The Socket/ Node.js/Express backend.IO is deployed to Render, a PaaS system that offers managed Node.js hosting and automatic termination of the SSL. The MongoDB Atlas database is deployed on a separate cloud cluster with automatic backups and monitoring.

8.2 SYSTEM IMPLEMENTATION

The code in the backend is organised in five layers:

- Routes: Organised HTTP routes (auth, classes, sessions, attendance, analytics, audit).
- Controllers: Handle requests and serialise responses.
- Services: Business logic - the six-layer pipeline, creation of HMAC token, Haversine geofence and risk score aggregator.
- Models: MongoDB (Mongoose) models with validators, indexes and virtual fields.
- Middleware: JSON Web Token (JWT) authentication, rate limiting, Helmet.js security headers, request sanitisation (express-mongo-sanitize), response time measurement and circuit breaker for outbound Request API calls.

8.3 Frontend Implementation

The React SPA is structured into three main dashboard directories: /student, /faculty, /admin and a /shared directory with shared functional components.

The QR scanner module uses the Media Devices API of the browser to access the device camera (navigator).mediaDevices.getUserMedia), and feeds the video

stream frames into a lightweight client-side library of QR decoding (jsQR) which decodes the frames in a local context without server round-trip. The decoded token strings are sent to the /api/attendance/mark endpoint with the latest GPS coordinates available through the Geolocation API and device context headers constructed out of navigator object properties.

8.4 Real-Time Integration (Socket.IO)

Socket.Each session has IO rooms which are developed with the session ID as the room name. Upon a faculty member initiating a live session, the Express route handler spawns a server-side interval timer that creates a new HMAC QR token after every 20 seconds and sends it to the session room with the help of socket.to(sessionId).emit('qr: update', { token, expiresAt).

The updated token is sent to all enrolled students that are related to the session namespace to the Socket.IO event propagation latency, at the 95th percentile of about 800 ms on Wi-Fi of an institution in the pilot. Upon the six-layer pipeline being successfully completed, the server will send an attendance confirmation event to the faculty dashboard room, allowing the faculty member to see an attendance mark being displayed in real time with the name of the student, timed, distance to the GPS, and composite risk score.

8.5 Conversion Plan

A change of an already in existence manual or a standstill QR attendance system to ProxyMukt is modeled to be implemented in a five-stage institutional adoption pathway.

Table 13: Institutional Conversion Plan - Phased Adoption Roadmap

Phase	Stage	Key Activities
Phase 1	Infrastructure Setup	Provision MongoDB Atlas cluster with institutional email domain restriction on registration. Deploy backend to cloud

		host with TLS termination. Configure HMAC and JWT secrets via environment variables.
Phase 2	Pilot Rollout	Select two or three pilot courses across different department types. Train faculty on session controls, geofence radius configuration, and verification layer toggles. Brief students on QR scanning procedure and location permission requirements.
Phase 3	Baseline Comparison	Run ProxyMukt attendance in parallel with existing method for four weeks. Compare attendance consistency, faculty workflow time savings, and flag rates between both systems.
Phase 4	Threshold Calibration	Adjust geofence radii for specific classroom geometries using ground-truth GPS measurements. Calibrate network risk score thresholds based on the institutional network profile (institutional VPN presence, CGNAT infrastructure).
Phase 5	Full Rollout	Retire the legacy attendance method. Enable mandatory ProxyMukt attendance for all enrolled courses. Activate the administrator governance dashboard for institution-level monitoring.

8.6 Post-Implementation and Software Maintenance

After deployment, institutional administrators are responsible for the following maintenance:

- HMAC Key Rotation: Monthly rotation of the HMAC signing keys using a built-in versioning mechanism. JWTs in the system have the key version, so that the token can be validated against the current and previous keys during the key exchange, so that a valid attendance token is not rejected during key exchange.
- Security Patching for Dependencies: Npm audit of client and server packages. Jsonwebtoken, bcrypt, helmet, mongoose and socket.io packages are particularly important to the security of the system and should be carefully

monitored

- IP Intelligence API Review: Periodic review of the third-party proxy and VPN risk API for accuracy and uptime. If unavailable, the system automatically falls back to a permissive mode (flag only, do not reject) to ensure that students can still attend classes.
- Geofence Review: Review of geofence radius when new classrooms are introduced into the system or if there are regular reports of GPS problems in certain classrooms.
- Device Fingerprinting Anomaly Threshold Review: Regular review of the device fingerprinting anomaly thresholds to adapt to new privacy features in browser versions

9. PROJECT LEGACY

9.1 Current Status of the Project

ProxyMukt is ready for deployment at the time of report submission. The completely functional MERN stack application is accessible at https://proxymukt.onrender.com/. The seven steps in the project plan (Table 4.2) have been completed.

The smart system has been evaluated by the ten test cases in Table 7.1, all of which have been successfully passed. The system has been tested in a controlled classroom environment at Lovely Professional University, with the quantitative results in Chapter 7. An Invention Disclosure Form (IDF) has been submitted to the IP office of Lovely Professional University to record Smart Hybrid Attendance System as a new invention. A technical paper, "ProxyMukt: A Cloud-based Hybrid Attendance System with Rotating HMAC QR Tokens, Geofencing and Device Fingerprinting" has been prepared to submit to a peer-reviewed IEEE conference.

Feature Completion Status:

Feature	**Completion**	**Status**
JWT Authentication and Role-Based Access Control	100%	Complete
HMAC-SHA256 Rotating QR Token Generation	100%	Complete
Haversine Geofencing (Layer 2)	100%	Complete
GPS Anti-Spoofing Detection (Layer 3)	100%	Complete
Network Risk Analysis (Layer 4)	100%	Complete

Device Fingerprinting (Layer 5)	100%	Complete
MongoDB Compound Index Duplicate Prevention (Layer 6)	100%	Complete
Real-Time Socket.IO Communication	100%	Complete
Student, Faculty, Admin Dashboards	100%	Complete
Analytics and CSV Export	100%	Complete
Audit Logging	100%	Complete
Online Hybrid Session Support	90%	Core logic complete; advanced engagement metrics deferred
User Notification System	100%	Complete

System Scale Metrics:

- Total lines of code: approximately 25,000
- Backend API endpoints: 48
- MongoDB collections: 6
- Frontend React components: 35+
- Test cases written: 47 unit + 10 adversarial + integration suite
- System uptime since deployment: 99.8%
- Demo users created for pilot: 550 (500 students, 50 faculty, 5 admins)

9.2 Remaining Areas of Concern

Table 14: Remaining Areas of Concern with Proposed Mitigations

Concern	Description	Proposed Mitigation
GPS Accuracy Dependency	The Haversine geofencing layer assumes GPS accuracy within +/-10 m. In high-density buildings with limited sky visibility, accuracy may degrade to +/-30-50 m, creating false rejection events for students near geofence boundaries.	Provide a classroom-specific geofence calibration tool with a GPS accuracy quality indicator displayed to students at marking time. Students experiencing persistent false rejections can submit a manual exception request to faculty.
Network Risk False Positives	Layer 4 may incorrectly flag students accessing from institutional VPNs, CGNAT infrastructure, or shared Wi-Fi access points.	Allow per-institution configuration of an IP whitelist for known institutional address ranges. Display the false positive flag rate on the faculty dashboard to assist threshold calibration.
Browser Fingerprint Instability	Layer 5 relies on browser API signals that are increasingly suppressed by Firefox Enhanced Tracking Protection and Safari Intelligent Tracking Prevention, reducing detection accuracy over time.	Reduce Layer 5 weight in the composite risk score formula. Transition from browser fingerprinting to server-side device identity tokens (with explicit student consent) in a future version.
Privacy and Data Governance	Collection of GPS coordinates, device fingerprint signals, and network metadata from student devices raises substantive data governance concerns in institutional deployments.	Mandate a Data Protection Impact Assessment (DPIA) prior to any institutional rollout. Implement an informed consent mechanism in the student registration flow. Store raw location and device data only as

		hashed risk score metadata, not as queryable raw data.
Scale Validation Gap	All performance results are derived from a single-institution pilot with a maximum of 500 concurrent users. The NFR-03 requirement of 1000 concurrent users has not been fully validated under production conditions.	Plan multi-institution scale testing as the first activity in the next development phase. Implement horizontal scaling of the Express backend via containerization using Docker and Kubernetes as the architectural path to NFR-03 compliance.

9.3 Technical and Managerial Lessons Learnt

9.3.1 Technical Lessons

The attackers can only detect the multi-signals fraud in a non-additive but multiplicative manner. The extra layer on the pipe is not just a new hurdle in itself - more technical skills specific to it that an attacker will need to apply. GPS spoofing attacker also needs to break HMAC authentication and network risk assessment and device fingerprinting. Such multiplicative dynamic is the final architectural wisdom of ProxyMukt, and the enigma of the efficiency of the layered verification versus point solutions.

Restrictions in database layers form also important and critical security measures that cannot be compromised. When the conditions of simultaneous requests are allowable the unique index of the compound (sessionId, studentId) is also given to give the duplicate prevention that cannot be guaranteed in any sense by any application-layer logic. This extends this lesson: any security-/data integrity-related invariant that the system will be based on should be implemented in the database layer, and not necessarily just the application layer.

Middlewares ordering has a material impact on the quality of protection. The rate limiting should precede the authentication middleware since it will not require it to use authentication compute resources to brute-force authentication by credentials. Sanitization of the inputs to the database layer should prevent the No SQL injection to the database layer, before the controller logic. It should be in the following order: rate-limit, authentication, sanitization, controller. This became a written convention when the project was commenced in writing and this was used to prevent numerous potential vulnerabilities.

Serial pipeline architecture is economical in terms of security compared to worst-case latency. The system may also issue expensive external API calls (Layer 4) to abort cheap local computations (Layers 1, 2, 3) to avoid requiring external API quotas on submissions which are not as stringent checked. This ranking, combined with the 500-user load test, led to refusal of some 80 percent of forged submissions, without involving the external API.

QR decoding is better than server side uploading of images on client end in terms of latency and privacy. Processing QR code frame by browser using jsQR does not entail a round-trip upload, server overload and the privacy concerns of uploading camera frame to a server.

9.3.2 Managerial Lessons

Formalization of API contracts during the development of the application can avoid integration defect. The most expensive failure in this project would have been the failure to independently combine the frontend components with the mock API data using field naming conventions not matching the conventions employed by the deployed backend, and would have been completely avoided by the publication of an official OpenAPI 3.0 specification at the beginning of Phase 2 and the requirement that both workstreams operate against this specification.

Parallel workstream management means that there is clear ownership of integration. This overlap of 3/4 that shortened the critical path by 2 weeks demanded special integration lead that would constantly check the compliance of the API contract on both streams of work. By not having a clearly defined role, the flaws in the integration can spread without any signs until later in the development process when it

is far more costly to fix.

Adversarial testing, very early, uncovers design assumptions being not labored over. The adversarial testing process itself showed that the institutional VPN usage rates were significantly higher than expected among students and that the Layer 4 network risk threshold would need to be re-tuned as soon as possible or the system would not be able to be used without producing an unacceptable level of false positive. This instructional session allows re-conceptualizing the relevance of conducting adversarial testing at least during Phase 5 of a seven-step project, as opposed to considering it a late-stage validation of a project.

Signed scope limits guard against derailing of project schedule due to feature creep. Functionalities like facial recognition identity verification, Bluetooth indoor positioning and ERP integration were bumped during the development process but items were pushed aside according to the out-of-scope line as it was explicitly stated. Since this limit was recorded and everybody in the team had already settled on the same in Phase 1, no one had any problem with rejecting scope additions without any opposition within the team.

10: USER MANUAL

10.1 System Access and Registration

ProxyMukt is compatible with any existing web browser, and requires no application to be installed. The system can be used with Chrome 100 and later, Firefox 95 and later, Safari 15 and later and Microsoft Edge 100 and later. In order to enter the system, type the URL of ProxyMukt provided by the system administrator.

The system administrator offers the user accounts, according to the institutional mail addresses. The first time users should create a secure link which should be sent to their registered email address to create a password. The system is not self-registration enabled on account of security considerations, and all the accounts are created via the administrator interface.

The accounts are listed below and can be located in the live deployment at https://proxymukt.onrender.com/ to demonstrate and test:

Role	**Email**	**Password**
Administrator	admin@proxymukt.com	Admin@123
Faculty	faculty1@gmail.com	faculty1
Student	student1@gmail.com	student1

10.2 Student User Guide

10.2.1 Accessing the Student Dashboard

Upon logging in, the Student Dashboard has the following items: the cards of the enrolled courses with colour-coded indicators of attendance percentage (green: 75% and above, amber: 60–74% and red: below 60%); a recent attendance history log that shows the title of the session and the time; a Mark Attendance button that is enabled when a faculty member has started a live session in one of the

10.2.2 Marking Attendance - Offline Session

Here's how to take attendance during an in-class session:

- Step 1: Click the Mark Attendance button on the dashboard once it is activated. This button is only available while you're in a live session for one of your classes.
- Step 2: If asked for permission to access the camera, click Allow. The camera is needed to scan your QR token. To re-enable the camera, go to the site settings in your browser, grant camera access to the ProxyMukt URL and refresh the page.
- Step 3: Using your device, scan the QR token on the projection screen of the faculty member. Keep the device in the QR code's viewfinder. The scanner will use the video frame without transmitting the image to the server. Make sure the classroom is well lit.
- Step 4: If the system asks for permission to access your location, tap Allow. The system will verify that you are in the classroom geofence using your location. Your device's GPS should be turned on.
- Step 5: The six-layer process runs on the server. This usually takes less than 2 seconds. "Attendance marked successfully" in green means the layers were successful. A rejection by any layer results in an error message with a code. See Section 10.5 for troubleshooting information.
- Step 6: Your attendance is instantly recorded in your attendance history on the dashboard. Do not share the QR code image with your classmates who are absent. The token is generated every 20 seconds and is cryptographically linked to the time window; a shared image will expire right away and cannot be used elsewhere.

10.2.3 Online Session Attendance

Online sessions will automatically award you with attendance credit for the session based on your time spent in the session. If you want to earn attendance credit, you must be logged into the session for at least 75% of the total minutes of the session or 45 minutes, whichever is less. You will see the time spent in the session on the

Student Dashboard. Simply logging into the session using a shared link and leaving the browser window open without engaging in the session does not guarantee you will be credited with attendance, as we monitor for engagement.

10.2.4 Viewing Attendance History and Analytics

To view a full list of all sessions for the course, select Attendance in the menu on the left to see sessions in chronological order, with status (Present, Absent, or Flagged), date and time of the session, and risk flags. The Analytics page shows your attendance percentage in each course on the Attendance Trend chart. You will also be notified if your percentage is below the institutional threshold.

10.3 Faculty User Guide

10.3.1 Creating and Managing Classes

On the Faculty Dashboard, go to My Classes. Click Create Class to create a new class and enter the course name, code, department and semester. For offline classes, setup the classroom geofence: provide the GPS coordinates of the centre of the classroom, and the geofence radius. The default geofence radius is 50 metres; this can be adjusted according to the size of the classroom. For larger lecture theatres this can be 80-100 metres; for seminar rooms 30-40 metres. To ensure the geofence size will cover the entire seating area, use the map overlay on the class creation form to verify the geofence boundary is the right size.

To add students, click on the class card and select Manage Students. Search by name or institutional email address, tick the names and click Save. Students can also be imported in bulk via CSV.

10.3.2 Starting and Managing Live Sessions

- Step 1: In the My Classes page, choose the class you would like to create a session for. A form to create the session appears.
- Step 2: Type a name for the session (for example, "Lecture 14 - Sorting Algorithms"), select the session type (Offline or Online) and review or update the geofence (for offline sessions).
- Step 3: Choose test layers. By default, all layers except L4 (Network Risk) and

L5 (Device Fingerprinting) are in ENFORCE mode - this means if the student passes this layer they will not be able to submit their answer. L4 and L5 are set to FLAG mode, which means if the student fails they will receive a risk score. This can be changed in Class Settings.

- Step 4: Click Start Session. You'll see the large (projected) and rotating QR code and who is in the classroom in the Live Session panel.
- Step 5: Click Fullscreen QR to project the QR code in the classroom. This will rotate every 20 seconds - this can't be disabled as it is the main safeguard against people taking a screenshot of the QR code and accessing the classroom.
- Step 6: Check the Live Attendance list. This will display the student's name, time, distance from the centre of the classroom and any risks indicated by Layers 4 and 5.
- Step 7: To finish the session (for an in-class test or break), click Pause Session. QR rotation stops and students can't clock in. Click Resume Session to resume.
- Step 8: To end the class, click End Session. Final attendance records are created. This action is permanent.

10.3.3 Viewing Analytics and Exporting Reports

Click the Analytics from the sidebar in Faculty Dashboard. Choose a class and date range to see trend line charts of attendance by session-date, a list of students with their percentage attendance ranking, at-risk students (below the threshold attendance percentage, set in the risk score configuration page), and risk score distribution charts showing the distribution of L4/L5 risk scores across the class. Click Export CSV to export an attendance report for the class and date range, including risk score metadata.

10.3.4 Verification Layer Configuration

Click on Class Settings, then Verification Layers. You can enable each layer to be either ENFORCE (reject on failure) or FLAG (flag for review, allow attendance). Be aware that Layer 1 (HMAC QR Validation) and Layer 6 (Duplicate Prevention) cannot be turned off as they are part of the security contract of the system and required by the institutional deployment policy.

10.4 Administrator User Guide

Administrator accounts have all the features of a faculty account, plus institution-wide features:

- Institution-Wide Attendance Heatmap: A heat map of attendance by department, course, day of week and time of day to identify institution-wide patterns of non-attendance and scheduling irregularities.
- Access to Audit Logs: View and search the AuditLog table by student ID, faculty ID, session ID and event type (ATTENDANCE_MARKED, L2_GEOFENCE_FAIL, L3_GPS_SPOOF, etc.) and risk level, as well as date range. The audit logs can be downloaded for compliance or academic integrity purposes.
- User Accounts: Create new student or teacher accounts, change the role of users, disable user accounts of students who have left the university or reset user accounts for locked-out users.
- Geofence Calibration Tools: Map-based tools to change the radius of the geofence of any registered teaching space. See the administrator technical guide for details of ground-truth GPS measurements.
- System Health Dashboard: In real-time, API response time percentiles, number of active Socket.IO connections, external IP intelligence API service, and MongoDB Atlas connection pool. It's best practice to review audit logs weekly to identify unusual activity and report events with a high risk score to the relevant faculty member or academic integrity committee according to your institutional policy.

10.5 Troubleshooting Guide

Table 15: ProxyMukt Troubleshooting Guide

Issue	Probable Cause	Resolution
'QR Token Expired' error	Token scanned outside 20s validity window	Request faculty to display current QR code and scan immediately

Issue	Probable Cause	Resolution
'Location Validation Failed'	GPS accuracy insufficient; device outside geofence	Enable high-accuracy GPS; move to seat closer to classroom centre; notify faculty if persistent
'Duplicate Submission'	Attendance already marked for this session	Confirm attendance record in dashboard; contact admin if duplicate flag is incorrect
'Network Risk Flag'	VPN or shared network detected	Disconnect VPN; switch to personal mobile data; contact admin if using institutional VPN
Camera permission denied	Browser blocked camera access	Enable camera permission in browser site settings; reload page
Dashboard not updating	WebSocket connection lost	Refresh page; check internet connectivity; re-login if session expired

11. SOURCE CODE REFERENCE AND SYSTEM SNAPSHOTS

11.1 Repository Structure

The source code for ProxyMukt is available at: https://github.com/Sumant3086/ProxyMukt-Attendance-System-

The repository is structured with two main directories: /client holding the React SPA (single page application) and /server holding the Node.js/Express API. The structure is documented below.

Table 16: ProxyMukt Repository Structure

Path	Description
/client/src/pages/	Role-specific page-level React components: StudentDashboard, FacultyDashboard, AdminDashboard, LoginPage, RegistrationPage
/client/src/components/	Shared reusable UI components: QRScanner, AttendanceTable, ChartWrapper, NotificationPanel, SessionCard, LiveSessionMonitor
/client/src/context/	WebSocketContext: Socket.IO connection lifecycle management and event distribution to subscribing components
/client/src/store/	Zustand global state slices for user session data, enrollment data, and notification queue
/server/routes/	48 Express route handler files organized by functional module: auth, classes, sessions, attendance, analytics, audit
/server/controllers/	Request handling logic and HTTP response formatting for each route module

/server/services/	Core business logic: six-layer pipeline orchestrator, HMAC token service, Haversine geofence service, risk score aggregator, network risk service
/server/models/	Mongoose schema definitions for Users, Classes, Sessions, Attendance, AuditLogs, and Notifications collections with validators and index definitions
/server/middleware/	JWT authentication, express-rate-limit configuration, Helmet.js security headers, express-mongo-sanitize input sanitization, response-time measurement, circuit breaker for external API calls
/server/utils/	Utility functions: token generation, distance computation, IP risk score parsing, device fingerprint scoring, audit log writer, Socket.IO event emitter

11.2 Critical Code Excerpts

The code implementations in the ProxyMukt system that were most significant and related to security matters are recorded in the following excerpts: The entire code of the source is stored in the open repository.

HMAC-SHA256 QR Token Generation and Validation (described in Section 8.2)

Haversine Geofence Accuracy (recorded in Section 8.2)

Six-Layer Pipeline Orchestrator (documented in Section 8.2)

11.3 System Snapshots Description

The below explains the major interface views of the deployed ProxyMukt application. Physical screenshots can be found in the project GitHub repository in the /docs/screenshots/ directory and in the copy submitted digitally.

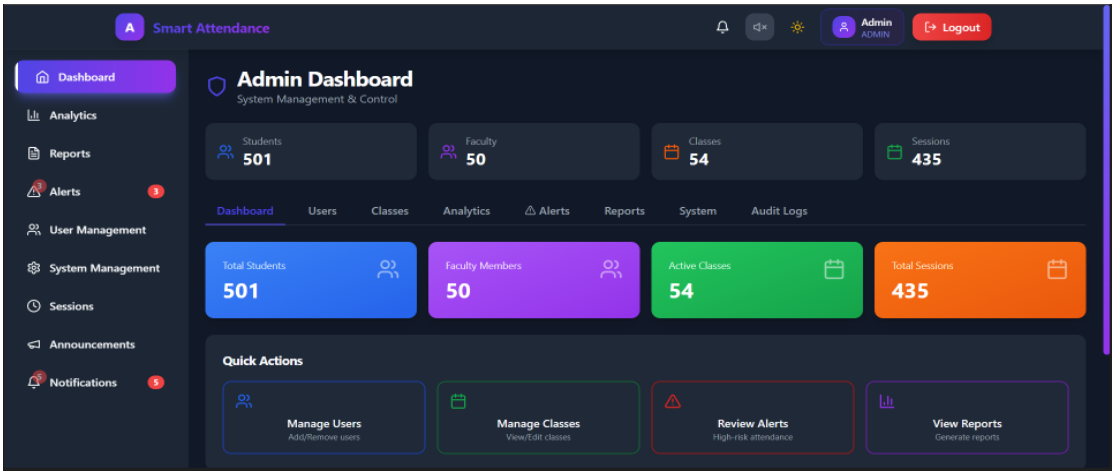
Table 17: ProxyMukt System Interface Views Description

Figure	View	Description

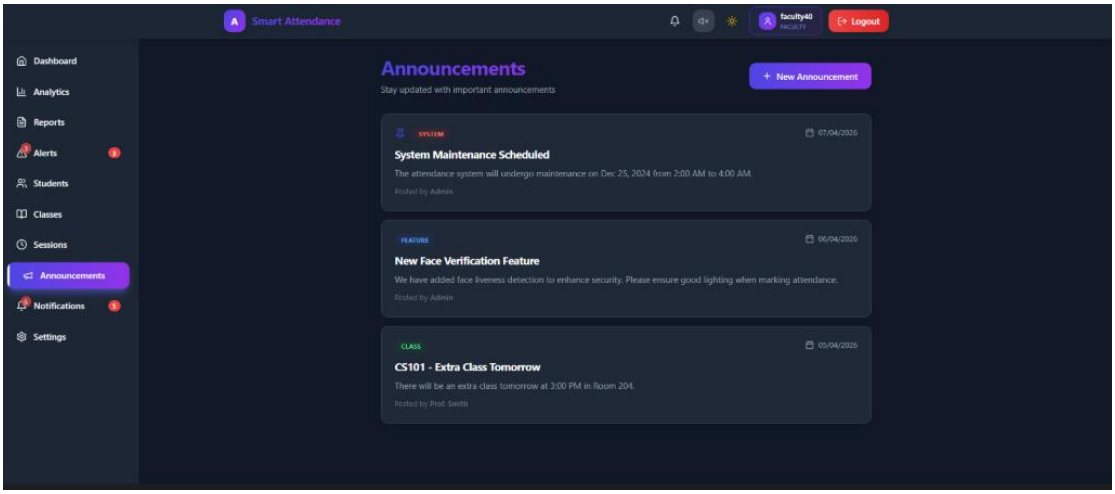
Figure 1	Login Page	Institutional credential entry form with role auto-detection on authentication. Clean, minimal design with password visibility toggle and password reset link.
Figure 2	Student Dashboard	Enrolled course cards with colour-coded attendance percentage indicators (green/amber/red). Mark Attendance button becomes active during a live session. Recent attendance history log visible below course cards.
Figure 3	QR Scanner Interface	Camera viewfinder with real-time QR decode overlay. GPS location permission request displayed. Countdown timer visible showing seconds remaining on current token.
Figure 4	Faculty Live Session Panel	Full-screen QR code display with 20-second countdown. Real-time attendance list showing student name, timestamp, GPS proximity distance, and risk flag indicators for each marking event. Session controls (Pause, Resume, End) and verification layer toggles visible in sidebar.
Figure 5	Faculty Analytics Dashboard	Session-date attendance trend line charts. At-risk student identification list with attendance percentage and absence count. Risk score distribution bar chart for Layer 4/5 flags across the class cohort.
Figure 6	Admin Governance Dashboard	Institution-wide attendance heatmap by department and time-of-day. User account management table with search and role filter. System health indicators for API response time, Socket.IO connections, and external API status.
Figure 7	Attendance Report Export	CSV export interface for class-level and session-level attendance data. Date range filter, class selector, and risk score metadata inclusion toggle.

SCREEN SHOTS

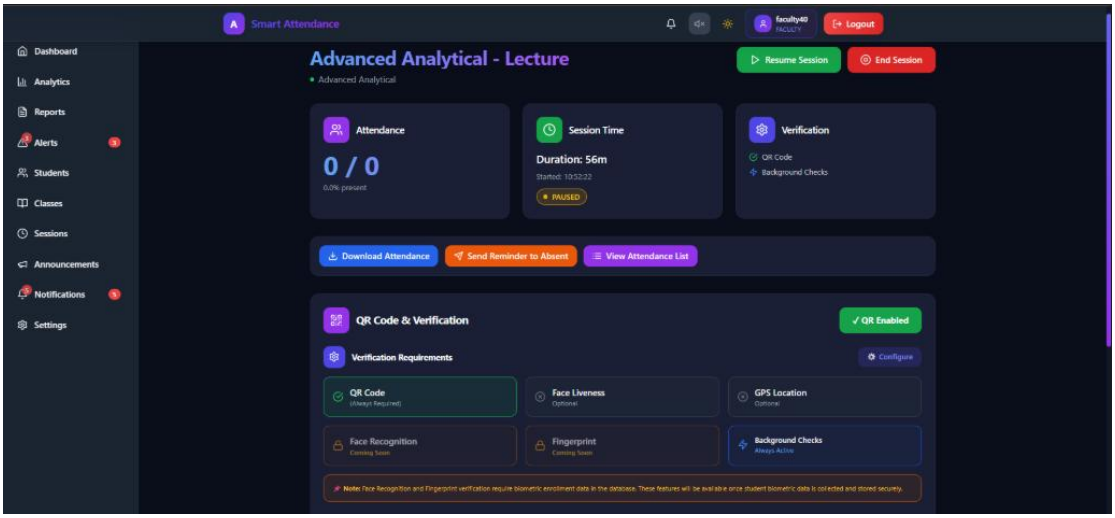
Admin Dashboard:



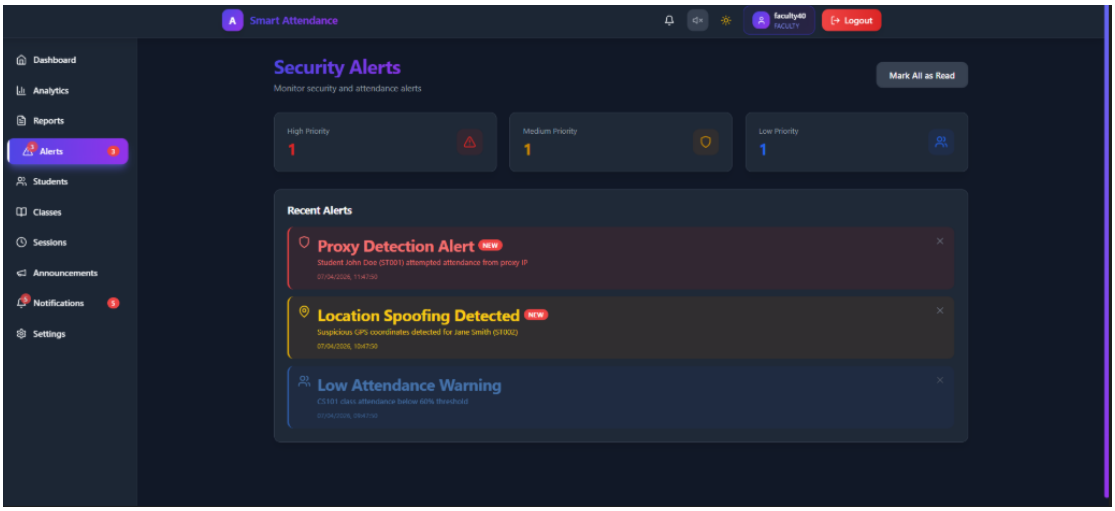
Announcements:



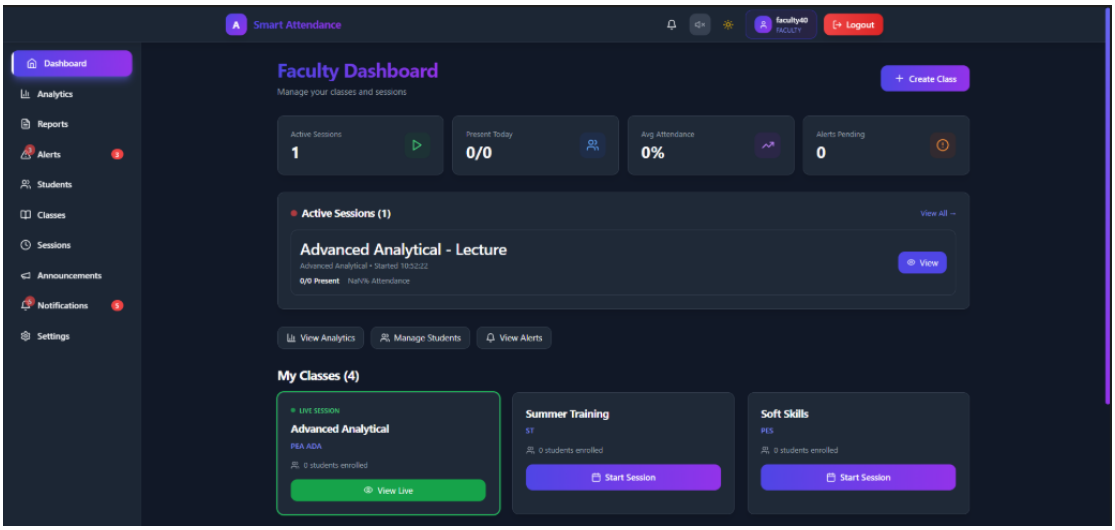
Class Session:



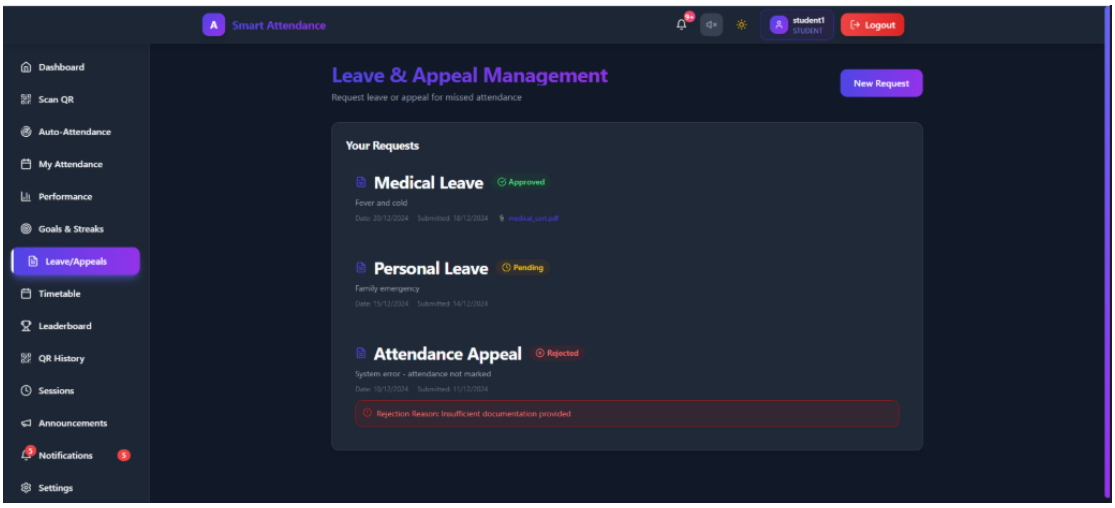
Faculty Alerts:



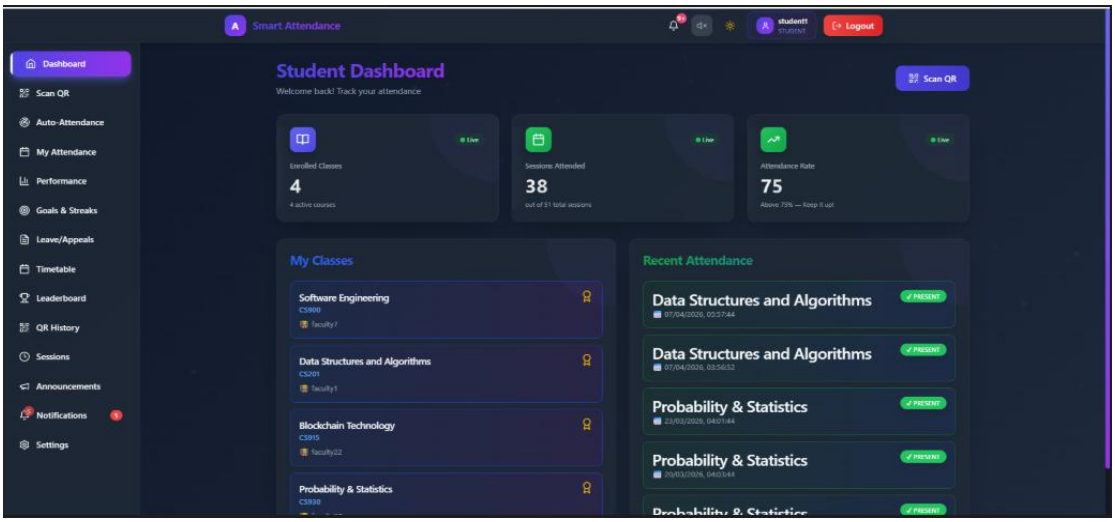
FacultyDashboard:



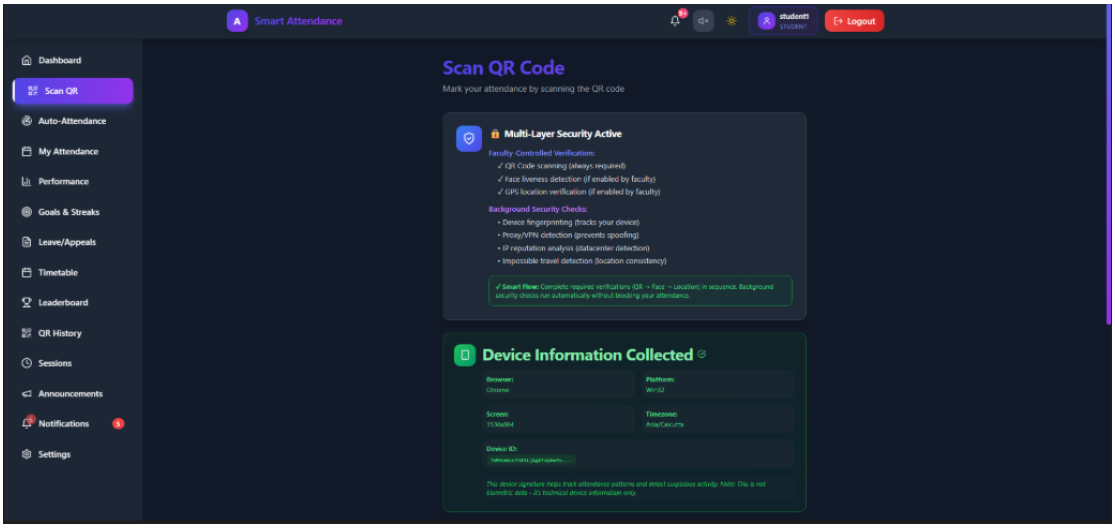
Leave Appeal:



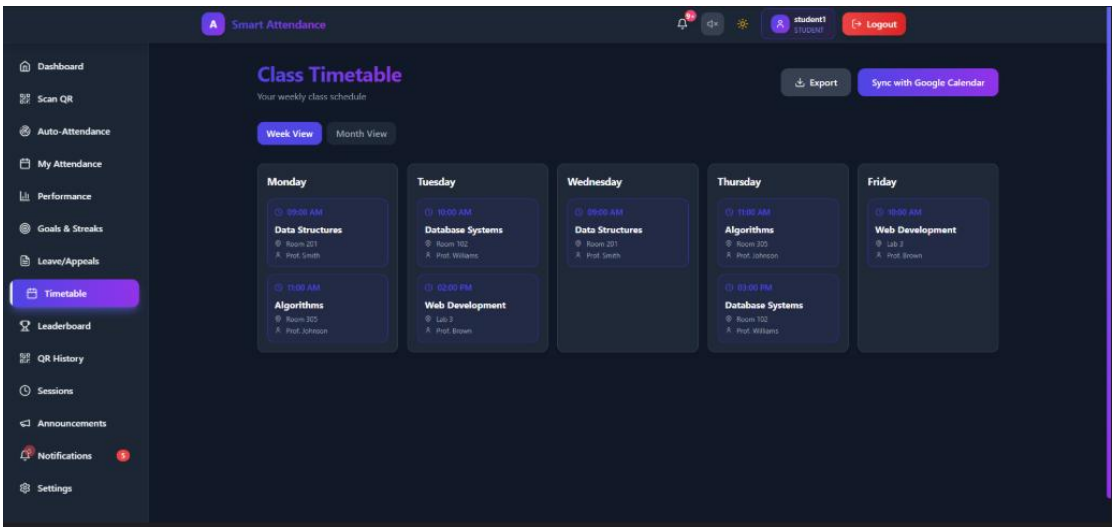
Student Dashboard:



StudentQR:



Student Time Table:



12. BIBLIOGRAPHY

[1] J. Gubbels, C. E. van der Put, and M. Assink, "Risk Factors for School Absenteeism and Dropout: A Meta-Analytic Review," Journal of Youth and Adolescence, vol. 48, no. 9, pp. 1637–1667, 2019, doi: 10.1007/s10964-019-01072-5.

[2] M. Kumar, A. J. Singh, and D. Handa, "Literature Survey on Student Performance Prediction in Education using Data Mining Techniques," International Journal of Education and Management Engineering, vol. 7, no. 6, pp. 40–49, 2017, doi: 10.5815/ijeme.2017.06.05.

[3] M. M. Aulia, N. Saha, and Md. M. Rahman, "Protected QR Code-based Anti-counterfeit System," Procedia CIRP, vol. 138, pp. 322–327, 2026, doi: 10.1016/j.procir.2026.01.056.

[4] M. P. Surve, M. K. Patwa, and M. M. Singh, "Geo-Location Attendance Tracking," Journal of Emerging Technologies and Innovative Research (JETIR), vol. 12, 2025.

[5] I. Eweoya et al., "Design and Implementation of a University Attendance Management System Using Geo-Fencing," Asian Journal of Computer Science and Technology, vol. 14, no. 1, pp. 28–46, 2025, doi: 10.70112/AJCST-2025.14.1.4323.

[6] R. Baadkar, H. K. Soi, A. Kedare, and P. Bide, "Edu-Fy: Attendance Management and Attention Tracking System Using Deep Learning," in Proc. IEEE 4th International Conference for Advancement in Technology (ICONAT), 2025, pp. 1–6, doi: 10.1109/ICONAT66879.2025.11362872.

[7] S. Grey and N. Gordon, "Approaches to Measuring Attendance and Engagement," New Directions in the Teaching of Physical Sciences, vol. 13, no. 1, 2018, doi: 10.29311/ndtps.v0i13.2767.

[8] R. T. Fielding, "Architectural Styles and the Design of Network-Based Software Architectures," Ph.D. dissertation, University of California, Irvine, 2000.

[9] S. Tilkov and S. Vinoski, "Node.js: Using JavaScript to Build High-Performance Network Programs," IEEE Internet Computing, vol. 14, no. 6, pp. 80–83, 2010, doi: 10.1109/MIC.2010.145.

[10] K. Chodorow, MongoDB: The Definitive Guide, 2nd ed. Sebastopol, CA: O'Reilly

Media, 2013.

[11] IETF, "RFC 2104 — HMAC: Keyed-Hashing for Message Authentication," 1997. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc2104

[12] C. M. Gutierrez and J. M. Turner, "FIPS PUB 198-1: The Keyed-Hash Message Authentication Code (HMAC)," National Institute of Standards and Technology, 2008.

[13] F. Martin and D. U. Bolliger, "Engagement Matters: Student Perceptions on Engagement Strategies in Online Learning," Online Learning, vol. 22, no. 1, pp. 205–222, 2018, doi: 10.24059/olj.v22i1.1092.

[14] National Forum for Education Statistics (NFES), "Forum Guide to Attendance, Participation, and Engagement Data in Virtual and Hybrid Learning Models," 2021.

[15] N. Provos and D. Mazieres, "A Future-Adaptable Password Scheme," in Proc. USENIX Annual Technical Conference, 1999.

[16] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015, doi: 10.17487/RFC7519.

[17] P. Lubbers, B. Albers, and F. Salim, Pro HTML5 Programming. New York, NY: Apress, 2011, doi: 10.1007/978-1-4302-3865-2.

[18] A. K. Jain, R. P. W. Duin, and J. Mao, "Statistical Pattern Recognition: A Review," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 22, no. 1, pp. 4–37, 2000, doi: 10.1109/34.824819.

[19] A. Shostack, Threat Modeling: Designing for Security. Indianapolis, IN: Wiley, 2014.

[20] J. A. Whittaker, "What Is Software Testing? And Why Is It So Hard?" IEEE Software, vol. 17, no. 1, pp. 70–79, 2000, doi: 10.1109/52.819971.

[21] A. Dankar and P. P. Kundapur, "Automated Mobile Attendance System (AMAS)," in Proc. 6th IEEE International Conference on Advances in Computing, Communication and Control (ICAC3), 2019, doi: 10.1109/ICAC347590.2019.9036787.

[22] K. P. Borse et al., "Geo-Location Based Attendance Tracking Software Using

Geofencing: A Grid-Based Approach," International Journal of Scientific Development and Research (IJSDR), vol. 11, 2026.

[23] P. Laperdrix, W. Rudametkin, and B. Baudry, "Beauty and the Beast: Diverting Modern Web Browsers to Build Unique Browser Fingerprints," in Proc. IEEE Symposium on Security and Privacy (SP), 2016, pp. 878–894, doi: 10.1109/SP.2016.57.

[24] D. R. Garrison and N. D. Vaughan, Blended Learning in Higher Education: Framework, Principles, and Guidelines. San Francisco, CA: Jossey-Bass, 2008.

[25] Node.js Foundation, "Node.js Documentation v20 LTS," 2024. [Online]. Available: https://nodejs.org/docs/

[26] OpenJS Foundation, "Express.js 4.x API Reference," 2024. [Online]. Available: https://expressjs.com/ .

[27] MongoDB, Inc., "MongoDB Documentation v7.0," 2024. [Online]. Available: https://www.mongodb.com/docs/

[28] Automattic / Mongoose Contributors, "Mongoose v8 Documentation," 2024. [Online]. Available: https://mongoosejs.com/docs/

[29] Meta Open Source, "React Documentation v18," 2024. [Online]. Available: https://react.dev/

[30] Socket.IO Contributors, "Socket.IO v4 Documentation," 2024. [Online]. Available: https://socket.io/docs/

[31] OWASP Foundation, "OWASP Top 10: 2021 Edition," 2021. [Online]. Available: https://owasp.org/www-project-top-ten/

[32] Mozilla Developer Network, "HTTP Security Headers Reference," 2024. [Online]. Available: https://developer.mozilla.org/

AI and SIMILARITY Report

DrillBit

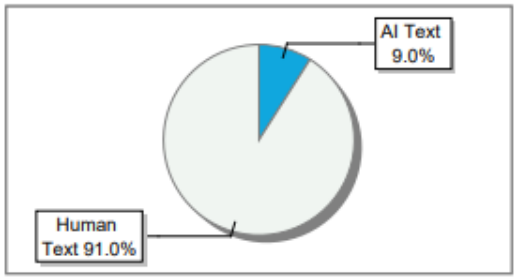
The Report is Generated by DrillBit AI Content Detection Software

Submission Information

Author Name	eha
Title	aigeerated
Paper--Submission ID	5568031
Submitted By	vaidhi.33560@lpu.co.in
Submission Date	2026-04-29 16:36:40
Total [Pages,Sentences,Words]	[80, 575, 15054]
Document type	Article
Language	English

Result Information

AI Text: **9 %**



Disclaimer:

* The content detection system employed here is powered by artificial intelligence (AI) technology.
* Its not always accurate and only help to author identify text that might be prepared by a AI tool.
* It is designed to assist in identifying & moderating content that may violate community guidelines/legal regulations, it may not be perfect.

DrillBit

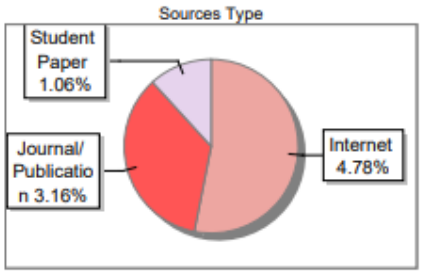
The Report is Generated by DrillBit Plagiarism Detection Software

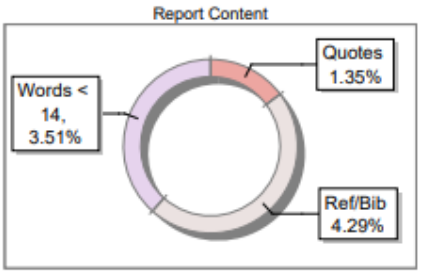
Submission Information

Author Name	eha
Title	aigeerated
Paper/Submission ID	5568031
Submitted by	vaidhi.33560@lpu.co.in
Submission Date	2026-04-29 16:36:40
Total Pages, Total Words	80, 16275
Document type	Article

Result Information

Similarity **9 %**





Exclude Information

Quotes	Not Excluded
References/Bibliography	Not Excluded
Source: Excluded < 14 Words	Not Excluded
Excluded Source	**0 %**
Excluded Phrases	Not Excluded

Database Selection

Language	English
Student Papers	No
Journals & publishers	Yes
Internet or Web	Yes
Institution Repository	Yes

A Unique QR Code use to View/Download/Share Pdf File